

UNIVERSITÀ DEGLI STUDI DI MILANO
FACOLTÀ DI SCIENZE E TECNOLOGIE

DIPARTIMENTO DI INFORMATICA
“GIOVANNI DEGLI ANTONI”



LAUREA TRIENNALE IN
SICUREZZA DEI SISTEMI E DELLE RETI INFORMATICHE

DESIGN E SVILUPPO DI UNA SOLUZIONE
PER LA VALUTAZIONE DI SISTEMI DISTRIBUITI

Autore: Enea Manzi
Matricola: 987326

Relatore: Prof. Marco Anisetti
Correlatore: Dr. Filippo Berto

A.A. 2023-2024

Abstract

Le moderne architetture distribuite, caratterizzate da decentralizzazione e dalla suddivisione in microservizi, pongono sfide rilevanti nella *valutazione del comportamento del sistema*. Data la complessità introdotta da tali architetture, vengono richiesti sistemi di monitoraggio avanzati in grado di verificare Proprietà Non Funzionale (PNF) per valutare il comportamento dell'infrastruttura. Le soluzioni attuali, sebbene efficaci nella valutazione di performance, risultano limitate quando si tratta di fornire una valutazione olistica e continua di tali sistemi, basata sulla verifica di *proprietà non funzionali* tramite *contratti formali e specifici*. È sorta quindi la necessità di sistemi di monitoraggio sempre più sofisticati, che possano raccogliere informazioni su molteplici aspetti del sistema in modo da garantire il soddisfacimento di specifiche proprietà.

Questa tesi, sfruttando una *metodologia di assurance* innovativa, mira a colmare alcune lacune nel monitoraggio e nella valutazione delle ormai comuni infrastrutture distribuite. La metodologia si basa su i) una raccolta continua e trasparente di *evidence*, o prove, che misurano stati rilevanti del sistema, tramite metriche che interrogano sistemi di monitoring dei nodi distribuiti, ii) la verifica formale di specifiche proprietà non funzionali, sulla base delle *evidence* raccolte con le metriche, attraverso contratti che ne definiscono i controlli da effettuare.

Viene proposta, basandosi su tale metodologia, un'integrazione per Elasticsearch tra le funzionalità dell'*Assurance Engine*, il sistema di monitoraggio e valutazione in corso di sviluppo, permettendo la raccolta di dati e la verifica di specifici e complessi contratti. Questo lavoro contribuisce quindi allo sviluppo di un sistema modulare flessibile, efficiente e scalabile per *verifiche di Assurance* basate su proprietà non funzionali, indipendentemente dalla complessità dell'infrastruttura sottostante.

I contributi di questa tesi sono: i) integrazione dell'*Assurance Engine* con Elasticsearch, sfruttando la metodologia adottata, ii) espressione di contratti complessi per valutare formalmente proprietà non funzionali su un target, iii) una valutazione sperimentale approfondita delle performance e dell'efficienza dell'integrazione sviluppata.

La tesi mostra una implementazione concreta della metodologia adottata con l'obiettivo di costruire un unico sistema centrale, efficiente e affidabile per monitorare sistemi distribuiti, l'*Assurance engine*, riducendo al minimo l'impatto sulle risorse computazionali. I risultati confermano che il sistema è in grado di fornire prestazioni efficienti nella raccolta di evidence tramite metriche e nella valutazione di contratti.

Ringraziamenti

Vorrei esprimere la mia sincera gratitudine a tutte le persone che mi hanno accompagnato durante questo percorso, contribuendo al raggiungimento di questo importante traguardo.

Desidero innanzitutto ringraziare il mio relatore, il Prof. Marco Anisetti, per la fiducia riposta in me e per avermi dato l'opportunità di svolgere questa esperienza.

Un sincero ringraziamento va al mio correlatore, il Dottor. Filippo Berto, per la sua passione e professionalità, per la costante disponibilità e per i preziosi consigli che hanno contribuito in modo significativo alla realizzazione di questo progetto, facilitandomi l'accesso a nuove conoscenze.

Desidero ringraziare di cuore i miei genitori. La loro presenza costante e il loro supporto incondizionato mi hanno accompagnato in ogni passo del mio percorso. I vostri sacrifici mi hanno permesso di raggiungere questi traguardi, e non smetterò mai di apprezzarli.

Un grazie speciale va alla mia fidanzata, Alice, che con la sua pazienza, comprensione e costante disponibilità mi ha affiancato e incoraggiato anche nei momenti più difficili. La sua vicinanza è stata per me un sostegno fondamentale, e le va la mia più profonda riconoscenza per aver alleggerito questo percorso.

Infine, ringrazio i miei amici per il supporto e il tempo trascorso insieme, vivendo momenti di leggerezza e compagnia preziosa.

Indice

Abstract	ii
Ringraziamenti	iii
Elenco delle figure	v
Elenco delle tabelle	vi
Elenco dei codici	vii
1 Introduzione	1
1.1 Gaps nella ricerca	1
1.2 Obbiettivi	2
1.3 Organizzazione della tesi	3
2 Background	5
2.1 Sistemi Distribuiti	5
2.1.1 Da sistemi monolitici a microservizi	7
2.1.2 Virtualizzazione e Container	8
2.1.3 Edge Cloud Continuum	11
2.2 Monitoring	13
2.2.1 Architettura di monitoring e relative applicazioni	14
2.2.2 Estrazione informazioni	15
2.3 Proprietà Non Funzionali	17
2.4 Verifica di Proprietà non Funzionali - Assurance	19
2.4.1 Assurance methodology	19
2.4.2 Assurance process	21
2.5 Certificazione di Proprietà non Funzionali - Certification	22
2.6 Related Work	24
3 Implementazione	27
3.1 Assurance Target	27
3.2 Experimental setup	28
3.3 ECK Stack	30
3.3.1 Deployment	31

Indice

3.3.2	ECK-stack deployment values	32
3.3.3	Funzionamento	34
3.4	Assurance Engine	36
3.4.1	Metric e Measurement	37
3.4.2	Contract ed Evaluation	39
3.4.3	API	40
3.4.4	Deployment e Vantaggi	43
3.4.5	Funzionamento	45
3.4.6	Esempi di richieste: Elastic e SQL Query	47
4	Esperimenti	50
4.1	Statistiche offerte	50
4.2	Benchmark	52
4.3	SEARCH Benchmark	54
4.3.1	Metric - <code>measure()</code>	54
4.3.2	Contract - <code>evaluate()</code>	56
4.4	SQL Benchmark	58
4.4.1	Metric - <code>measure()</code>	59
4.4.2	Contract - <code>evaluate()</code>	60
5	Discussione	63
5.1	SEARCH Benchmark	63
5.2	SQL Benchmark	64
5.3	Valori anomali	64
6	Conclusioni	67
6.1	Metodologia e Funzionamento	67
6.2	Esperimenti e risultati	68
6.3	Future Work	69
	Bibliografia	72

Elenco delle figure

Figura 2.1	Panoramica sull'infrastruttura Edge Cloud	12
Figura 2.2	Schema di <i>assurance methodology</i>	20
Figura 2.3	Schema di <i>assurance process</i>	21
Figura 2.4	Generico processo di certificazione	23
Figura 3.1	Diagramma del flusso di funzionamento dello stack	35
Figura 3.2	Diagramma del funzionamento	36
Figura 3.3	Interfaccia di Swagger API esposte	41
Figura 3.4	Interfaccia di Swagger esempio richieste POST	42
Figura 3.5	Interfaccia di Swagger schema json con i tipi associati ai valori	42
Figura 3.6	Interfaccia di Swagger schema risposte POST	43
Figura 3.7	Diagramma Assurance Engine deployato su 3 nodi	44
Figura 3.8	Assurance Engine: Flusso di esecuzione	46
Figura 4.1	Directory tree generato dai benchmark di Criterion	53
Figura 4.2	Diagramma a violino richieste di tipo SEARCH	54
Figura 4.3	Diagramma densità probabilistica richieste measure di tipo SEARCH	55
Figura 4.4	Diagramma iteration/time richieste measure di tipo SEARCH	56
Figura 4.5	Diagramma densità probabilistica richieste evaluate di tipo SEARCH	57
Figura 4.6	Diagramma iteration/time richieste evaluate di tipo SEARCH	58
Figura 4.7	Diagramma a violino richieste di tipo SQL	59
Figura 4.8	Diagramma densità probabilistica richieste measure di tipo SQL	59
Figura 4.9	Diagramma iteration/time richieste measure di tipo SQL	60
Figura 4.10	Diagramma densità probabilistica richieste evaluate di tipo SQL	61
Figura 4.11	Diagramma iteration/time richieste evaluate di tipo SQL	62
Figura 5.1	Diagramma confronto densità probabilistica	65
Figura 5.2	Diagramma confronto iteration/time	66

Elenco delle tabelle

Tabella 3.1	Risorse disponibili nel sistema	28
Tabella 4.1	Valutazioni statistiche SEARCH/Metric benchmark	55
Tabella 4.2	Valutazioni statistiche SEARCH/Contract benchmark	57
Tabella 4.3	Valutazioni statistiche SQL/Metric benchmark	60
Tabella 4.4	Valutazioni statistiche SQL/Contract benchmark	61

Elenco dei codici

3.1	Elasticsearch yaml values	32
3.2	Kibana yaml values	32
3.3	Kibana Policy yaml values	33
3.4	Elastic Agent yaml values	34
3.5	Fleet Server yaml values	34
3.6	Metric Trait	37
3.7	Arguments Struct	37
3.8	Measurement Struct	38
3.9	Output Struct	38
3.10	Contract Trait	39
3.11	Argument Contract Struct	39
3.12	Evaluation Struct	40
3.13	Valori JSON per query <i>match all</i>	47
3.14	Valori JSON per query <i>range</i> su intervalli temporali	48
3.15	Valori JSON per query <i>SQL</i>	48
3.16	Esecuzione Rust della query nel linguaggio di Elastic	48
3.17	Esecuzione Rust della query SQL	49

Capitolo 1

Introduzione

L'evoluzione delle tecnologie negli ultimi decenni, a causa dei limiti delle singole macchine, ha spinto il passaggio da architetture monolitiche e centralizzate ad architetture distribuite, caratterizzate dalla presenza di nodi interconnessi, fondate su microservizi che garantiscano funzionalità e prestazioni richieste. Data la complessità di questi nuovi standard architetturali, sorge spontanea la necessità di un adeguatamente complesso sistema di monitoring atto a collezionare *evidence* dai vari nodi, con lo scopo di stabilirne lo stato di salute e valutarne il comportamento rispetto determinate proprietà. Si passa quindi da sistemi di monitoring completamente centralizzati a sistemi distribuiti, che permettono l'esecuzione di metriche per la raccolta di prove sui vari nodi della rete.

Per soddisfare le esigenze di controlli sempre più completi sui sistemi e sul loro stato sono state introdotte le PNF, si tratta di proprietà che definiscono come il sistema svolge le sue funzionalità operative. É qui che entrano in gioco le tecniche di *Assurance*, il cui scopo è garantire il soddisfacimento di determinate PNF. Si tratta di tecniche che partono dalla raccolta di evidence tramite metriche e le sottopongono ad una valutazione tramite contratti formali, i quali consentono di verificare se le PNF vengono soddisfatte.

1.1 Gaps nella ricerca

Le soluzioni attualmente disponibili per effettuare verifiche di assurance tramite contratti, sebbene avanzate, presentano ancora alcune lacune che, se colmate, renderebbero il sistema più completo, versatile ed efficace. Tra queste mancanze si possono evidenziare alcuni *gaps*:

G₁ Generalizzabilità I sistemi di assurance attualmente in uso tendono a essere specifici per certi contesti o domini applicativi, limitando la loro applicabilità a scenari più ampi. La mancanza di un framework generalizzabile ostacola l'adozione di queste solu-

zioni in ambienti diversi, come quelli dinamici o su larga scala. Un sistema di assurance dovrebbe essere in grado di adattarsi flessibilmente a vari contesti mantenendo però elevati livelli di precisione ed efficienza. Questo gap evidenzia la necessità di creare un framework che si integri con strumenti di monitoring senza compromettere le prestazioni o l'affidabilità complessiva del sistema.

G₂ Valutazioni performance Molte delle soluzioni esistenti concentrano i loro sforzi esclusivamente sulle proprietà legate alle performance, come ad esempio latenza, *throughput* e utilizzo delle risorse, trascurando altre PNF altrettanto importanti, come la scalabilità, l'affidabilità o la disponibilità. Questo focus ristretto limita la capacità del sistema di fornire un quadro completo sul target. Occorre quindi sviluppare contratti che tengano conto di un insieme più ampio di proprietà, permettendo di verificare non solo le performance, ma anche altre PNF chiave del sistema.

G₃ Regole Avanzate per Contratti I sistemi attuali sono pressoché incapaci di supportare contratti complessi basati su regole avanzate per svolgere verifiche di assurance. I contratti utilizzati tendono a essere troppo semplici o rigidi per adattarsi efficacemente a situazioni variabili o scenari complessi, come quelli presenti in ambienti distribuiti o dinamici. La mancanza di una tale possibilità impedisce una valutazione accurata e adattiva delle PNF del sistema.

1.2 Obiettivi

La tesi si articola attorno a una serie di obiettivi di ricerca atti a migliorare il sistema di assurance attualmente in sviluppo, che possono essere riassunti come segue:

O₁ Estensione *Assurance Engine* Estensione di un framework di monitoring generico e altamente scalabile per sistemi distribuiti basato su verifiche di assurance tramite contratti. Si vuole integrare il sistema attualmente disponibile con un nuovo applicativo di monitoring, lo stack di Elasticsearch, permettendo il salvataggio di dati di monitoring e la ricerca tramite query. Il sistema integrato sarà in grado di raccogliere dati da fonti diverse tramite metriche e utilizzarli per valutare la conformità a contratti definiti formalmente, per la valutazione di PNF. Per effettuare queste valutazioni, vengono estratti dati da Elasticsearch o da altre sorgenti sotto forma di *measurements*, utilizzandoli come *evidence* o prove nella valutazione dei contratti definiti.

O₂ Implementazione Contratti I contratti definiscono le modalità per verificare la correttezza delle proprietà PNF in questione e sono strutturati come regole applicabili

a serie temporali di misurazioni. La validazione di un contratto avviene quando le *measurements*, raccolte tramite metriche e utilizzate come prove, confermano che le regole stabilite sono rispettate, garantendo così determinate proprietà del sistema. Si vogliono implementare contratti complessi per eseguire valutazioni approfondite sui dati raccolti.

O₃ Valutazione Sperimentale Performance La valutazione sperimentale del nostro sistema per verifiche di assurance per sistemi distribuiti è stata condotta in un ambiente di test circoscritto, utilizzando il cluster Kubernetes del laboratorio. L'obiettivo è stato quello di analizzare e valutare le performance operative del sistema.

1.3 Organizzazione della tesi

La tesi è suddivisa nei seguenti capitoli:

Nel **Capitolo 2**, viene introdotto il contesto teorico dei sistemi distribuiti, con un focus particolare sull'evoluzione dalle architetture monolitiche a quelle basate su microservizi, l'uso di tecnologie di virtualizzazione e containerizzazione, e infine l'Edge Cloud Continuum come possibile scenario da monitorare. Viene introdotto anche il concetto di PNF come un aspetto da verificare all'interno di un sistema distribuito. Si approfondiscono infine i concetti di monitoraggio nei sistemi distribuiti tramite tecniche di *Assurance* e *Certification*, con particolare attenzione alla metodologia adottata.

Nel **Capitolo 3**, viene descritto nel dettaglio il processo di implementazione della soluzione di monitoraggio proposta. Si propone una descrizione iniziale sul cluster kubernetes utilizzato per il deployment così da favorirne la replicabilità. Successivamente viene illustrato il deployment e il funzionamento dello stack di Elasticsearch e delle sue componenti all'interno del cluster. Infine, viene introdotto l'*Assurance Engine*, il componente sviluppato il cui obiettivo è quello di valutare proprietà non funzionali utilizzando metriche e contratti; è possibile interagire con il sistema tramite le Application Programming Interface (API) esposte. Viene descritto il motivo della sua realizzazione e i vantaggi che offre, il funzionamento del sistema e degli esempi di esecuzione.

Nel **Capitolo 4**, vengono illustrati gli esperimenti condotti per testare la soluzione implementata. Vengono eseguiti 2 benchmark, il primo basato su query nel linguaggio di Elasticsearch e il secondo basato su query SQL; per ciascuno vengono valutate le performance di esecuzione di metriche e contratti. I risultati dei benchmark vengono poi commentati e riportati tramite diagrammi e tabelle.

Capitolo 1 Introduzione

Nel **Capitolo 5**, vengono analizzati e discussi i risultati ottenuti durante la fase sperimentale cercando di spiegare alcuni picchi di latenza ottenuti nelle misurazioni e le differenze temporali tra `measure()` ed `evaluate()`. Il dettaglio si pone sulla spiegazione delle statistiche e delle oscillazioni ottenute.

Nel **Capitolo 6**, si traggono le conclusioni finali della tesi, sintetizzando i risultati raggiunti e valutando come questi rispondano alle sfide identificate inizialmente. Si propongono inoltre possibili direzioni di sviluppo futuro, con focus su scalabilità, nuove integrazioni e applicazioni in ambienti reali.

Capitolo 2

Background

Nel contesto delle moderne infrastrutture distribuite, il monitoraggio e la verifica di proprietà svolgono un ruolo cruciale nel garantire l'affidabilità, la sicurezza, il comportamento desiderato e le performance dei sistemi complessi. Con l'evoluzione delle tecnologie come la virtualizzazione, la containerizzazione e l'adozione di architetture a microservizi, la necessità di soluzioni di monitoring efficaci è cresciuta esponenzialmente. Tali soluzioni devono essere in grado di raccogliere, analizzare e valutare dati provenienti da vari nodi distribuiti, al fine di mantenere la continuità operativa e prevenire eventuali guasti. Questo capitolo esplora in dettaglio il concetto di monitoring nei sistemi distribuiti analizzando le principali *proprietà non funzionali* e focalizzandosi sulle metodologie di *Assurance* e *Certification* basati su *metriche, evidence e contratti*

Le tecniche di **assurance** sono finalizzate a garantire che un sistema rispetti, e quindi soddisfi, determinate *proprietà non funzionali* precedentemente definite.

La **certification**, invece, consiste nel processo di verifica che attesta la conformità del sistema a requisiti specifici.

2.1 Sistemi Distribuiti

I paradigmi architetturali tradizionali si basano su un sistema centralizzato, caratterizzato da un singolo nodo centrale con cui tutti gli utenti interagiscono, che coordina tutte le operazioni e fornisce dei servizi, tuttavia questa tipologia di architettura non è più in grado di soddisfare le esigenze del mercato odierno, richiedente elaborazione in tempo reale di grandi quantità di dati e interazione *real-time* per un numero crescente di utenti. Per cercare di soddisfare queste esigenze hanno avuto rapida diffusione i *sistemi distribuiti*, caratterizzati dalla presenza di diversi nodi interconnessi, non obbligatoriamente nello stesso luogo fisico, ma che operano come un'unica unità agli occhi dell'utilizzatore suddividendo così compiti e risorse. Lo scopo di questi sistemi è gestire in modo efficient-

te le risorse a disposizione, elaborare rapidamente grosse quantità di dati e fornire agli utenti dei servizi.

I vantaggi nell'utilizzo di sistemi distribuiti risiedono nelle migliori apportate ad affidabilità e prestazioni. Rispettivamente, non esiste più *il singolo punto di errore* centrale ma vari nodi distribuiti (con anche repliche) pronti a sostituire il nodo guasto; mentre la seconda migliona emerge dalla facilità con cui un sistema e i relativi nodi sono *scalabili* sia orizzontalmente (creazione di altri nodi) che verticalmente (assegnazione di più risorse ad un nodo) andando così a mantenere stabili le prestazioni. Questa architettura non solo migliora la scalabilità e l'affidabilità dell'applicazione, avendo diversi nodi distribuiti e sempre attivi, ma offre anche una maggiore flessibilità e resilienza di fronte ai guasti.

I sistemi distribuiti, data la loro complessità realizzativa, richiedono di soddisfare determinate *proprietà* per non vanificare gli sforzi implementativi e valorizzare l'impiego di questa architettura [1]. Tra questi abbiamo:

- *Accessibilità delle risorse*: un sistema distribuito deve, come obiettivo primario, rendere le risorse remote facilmente accessibili e amministrarle in modo efficiente.
- *Trasparenza*: un sistema distribuito deve nascondere il fatto che le risorse e i processi sono fisicamente distribuiti nella rete su vari sistemi, e deve presentarsi agli utenti come un'unica entità.
- *Apertura*: un sistema distribuito deve offrire servizi fondati su regole standard che ne descrivono la sintassi e la semantica. I servizi offerti vengono specificati attraverso *interfacce* definite con un *Interface Definition Language* come OpenAPI¹ (usato da Swagger²).
- *Scalabilità*: un sistema distribuito deve, in base alle esigenze, aumentare o diminuire le risorse e i nodi; è la proprietà che differenzia in modo sostanziale un sistema distribuito da uno centralizzato e ne consente la facile evoluzione.

I *sistemi distribuiti* vengono anche classificati in 3 macro-aree secondo [1]:

- *Computing systems* (sistemi di elaborazione): utilizzati per attività di elaborazione ad alte prestazioni; vedono il loro successo quando il rapporto prezzo/performance dei singoli dispositivi non diventa più accessibile ma conviene costruire un sistema componendolo.

¹<https://openapi.it/>

²<https://swagger.io/>

- *Information systems* (sistemi informativi): utilizzati in organizzazioni che possiedono una grossa quantità di applicazioni in rete per semplificarne l'interoperabilità tramite la comunicazione diretta delle varie componenti; questa procedura viene definita *enterprise application integration*.
- *Pervasive/embedded systems* (sistemi pervasivi/integrati): sono i sistemi caratterizzati da dispositivi piccoli, alimentati a batteria, mobili e con connessioni wireless, comunemente identificati come *smartphones*. Un importante vantaggio di questi sistemi è l'assenza di supervisione amministrativa umana: i dispositivi devono scoprire automaticamente e autonomamente il contesto e ambientarsi per poter funzionare.

2.1.1 Da sistemi monolitici a microservizi

Un sistema **centralizzato o monolitico**, modello tradizionale di deployment degli ultimi decenni, consiste in un applicativo software compilato come unica entità, autonoma e indipendente in grado di fornire dei servizi. Lo stato è contenuto all'interno dell'unico nodo centrale.

Questo approccio, sebbene semplice da implementare e gestire inizialmente, presenta diversi limiti man mano che l'applicazione cresce in termini di dimensioni e complessità [2]; tra le principali limitazioni vengono identificate le seguenti:

- *Flessibilità e Scalabilità limitate*: le modifiche, sia funzionali che di scalabilità, devono essere apportate su tutto il sistema, essendo costituito da un'unica entità, e non sulla singola unità operativa che lo necessita. I requisiti di prestazioni computazionali che una singola macchina dovrebbe soddisfare per permetterne un adeguato scaling verticale potrebbero risultare insoddisfacibili.
- *Affidabilità limitata*: essendo il sistema costituito da un'unica entità, *single-point-of-failure*, un singolo errore potrebbe compromettere l'intero sistema oppure fare da *collo di bottiglia*.
- *Tempi di sviluppo e deployment lenti*: la natura monolitica impone di dover ricompilare e ridistribuire l'intera applicazione anche per piccole modifiche, rallentando il ciclo di sviluppo e rilascio.

La transizione da architetture monolitiche a **microservizi** rappresenta una delle evoluzioni più significative nel campo dello sviluppo software degli ultimi decenni segnando l'inizio del Service-Oriented Computing (SOC), il cui obiettivo è usare i servizi come struttura base per costruire applicazioni [3]. Questa trasformazione nasce dall'esigenza di trovare una soluzione alle restrizioni presenti nei precedenti modelli architetturali.

In un'architettura a microservizi, un'applicazione è suddivisa in una serie di servizi mo-

dulari indipendenti, ciascuno dei quali è responsabile di una specifica funzionalità e può essere sviluppato, distribuito e scalato in modo autonomo. Questa metodologia si integra perfettamente con il concetto di *Software as a Service (SaaS)* e come architettura modulare per componenti Internet of Things (IoT), garantendo guadagni in termini di performance, dinamismo e resilienza [4] [5]. I vantaggi principali derivanti dall'adozione di questa architettura sono:

- *Scalabilità granulare*: ogni servizio può essere scalato indipendentemente dagli altri andando quindi ad ottimizzare l'utilizzo di risorse risultando in un complessivo aumento di performance
- *Flessibilità nello sviluppo*: lo sviluppo rispecchia la struttura isolata dei vari applicativi permettendo manutenzioni separate per ogni servizio
- *Ciclo di Sviluppo e Deployment rapidi*: ogni microservizio viene distribuito come entità indipendente dalle altre accelerando il rilascio di nuove funzionalità (*continuous delivery*)
- *Rafforzamento dell'affidabilità*: i guasti sono strutturalmente isolati a livello di servizio riducendo le possibilità di blocco dell'intero sistema.

Le *architetture a microservizi* sono diventate uno standard architetturale nello sviluppo di sistemi distribuiti e si integrano particolarmente bene a tecniche di *continuous delivery* con periodici rilasci automatici, rientrando quindi nella categoria del *DevOps* [6]. Vengono eseguiti maggiori rilasci di aggiornamenti grazie alla divisione delle componenti e alla possibilità di lavorare su singole parti dell'applicativo.

2.1.2 Virtualizzazione e Container

I sistemi distribuiti, caratterizzati dalla cooperazione di più nodi per eseguire compiti complessi, hanno guidato l'evoluzione verso tecnologie che ottimizzano l'efficienza e la scalabilità, come la virtualizzazione e la "containerizzazione". La prima consente di sfruttare al massimo le risorse hardware, creando ambienti isolati che eseguono applicazioni in parallelo. La seconda, evoluzione del primo concetto, offre un livello di isolamento più leggero e rapido, permettendo una distribuzione agile delle applicazioni in ambienti distribuiti. La sinergia tra sistemi distribuiti e queste tecnologie è fondamentale per l'implementazione efficiente e scalabile di architetture moderne.

Virtualizzazione il processo di virtualizzazione consiste nel creare un livello di astrazione sopra a quello fisico del sistema, rendendo così disponibili risorse in forma software e permettendo la suddivisione del singolo sistema in molteplici virtuali con le rispetti-

ve risorse allocate. Il sistema che ospita la virtualizzazione, comunemente identificato come “host”, condivide le proprie risorse hardware con i molteplici sistemi software che ospita, comunemente identificati con “guests”. Il software che viene utilizzato per raggiungere questo scopo è definito come “hypervisor”. Ogni sistema virtuale viene creato in un ambiente di lavoro completamente isolato contenente il sistema operativo, i file e le librerie necessarie; questa procedura provoca un overhead nel sistema e richiede degli istanti di tempo prima che la macchina sia avviata ma dall’altro lato garantisce una forte proprietà di isolamento: una violazione di sicurezza in un sistema (guest, non host) non compromette gli altri sistemi, è invece possibile compromettere l’hypervisor. La proprietà di portabilità è garantita ma sono presenti alcuni vincoli legati all’hardware sottostante.

Containerizzazione dato il trend degli ultimi anni verso ambienti virtuali in cloud, ha preso piede questo concetto introducendo una valida e più leggera alternativa alla classica virtualizzazione completa di un sistema operativo. A differenza della virtualizzazione completa, ogni applicazione containerizzata condivide il kernel del sistema host ma mantiene isolate le relative variabili d’ambiente, librerie e processi; in questo modo l’overhead generato è minimo e il tempo di avvio è rapido (non richiede l’esecuzione di un intero sistema operativo). Queste caratteristiche portano alla proprietà di flessibilità, scalabilità e facilità di gestione. Rispettivamente la facilità di avviare e fermare un container, la facilità di aumentare e diminuire le risorse necessarie in un certo istante di tempo e la facilità con cui è possibile effettuare manutenzione come aggiornamenti sull’applicazione. Permette anche la creazione di un ambiente altamente riproducibile, contenente l’applicazione e tutte le dipendenze, che potrà essere messo in atto su un qualsiasi sistema che esegua una container engine, indipendentemente dalle specifiche. Dall’altro lato si evince che, condividendo il sistema operativo dell’host, si ha una maggiore probabilità che una violazione in un container possa diffondersi ad altri.

Un altro aspetto cruciale per l’adozione dei container come standard è legato alla transizione da applicazioni monolitiche ad architetture basate su microservizi, come illustrato nella Sezione 2.1.1, che vedono la loro concretizzazione deployando ogni servizio in un container. In questo nuovo paradigma, ogni applicazione è composta da una serie di servizi piccoli e indipendenti che collaborano tra loro. La gestione e il deployment sono stati altamente velocizzati e semplificati, garantendo una maggiore affidabilità grazie alla possibilità di sviluppare singoli servizi separati in container.

Docker e Kubernetes

Le tecnologie che hanno assunto il ruolo di leader nel settore della virtualizzazione e della containerizzazione sono *Docker* e *Kubernetes*; il primo pone le basi per lo sviluppo

di applicazioni in ambienti isolati mentre il secondo si occupa di fornire delle migliorie ad una tecnologia ormai consolidata.

Docker³: è una piattaforma open-source che consente di automatizzare la distribuzione di applicazioni all'interno di container, ambienti isolati e portatili. Un container, istanza di una specifica immagine, include tutto il necessario per eseguire un'applicazione, come il codice, le librerie e le dipendenze, garantendo che l'applicazione funzioni in modo coerente su diverse infrastrutture, fornendo quindi replicabilità, portabilità e compatibilità.

Kubernetes⁴: è un progetto open-source diffusamente utilizzato in concomitanza con Docker in quanto semplifica notevolmente diverse operazioni nella gestione di container in ambienti distribuiti. Il suo scopo è quello di automatizzare la gestione, il deployment e il ridimensionamento di applicazioni containerizzate fornendo funzionalità come *orchestrazione*, *bilanciamento del carico*, *scalabilità* delle applicazioni e *rollout/rollback automatizzati*, le quali migliorano efficienza, affidabilità e disponibilità in ambienti di produzione complessi.

Con *orchestrazione* si intende il processo di gestione automatizzata di container su larga scala all'interno di un cluster di nodi. Questo include l'automazione di compiti complessi come il posizionamento dei container sui nodi più appropriati, il monitoraggio dello stato dei container, il riavvio di quelli che falliscono (self-healing), la gestione delle risorse di sistema (load-balancing) e la gestione delle interazioni tra i vari componenti dell'applicazione.

- *Network Load balancing*: processo di distribuzione uniforme del traffico di rete in ingresso tra più istanze di un'applicazione o servizio, eseguite in diversi container. Avviene attraverso l'utilizzo di Service, che astraggono set di Pod (gruppi di container, la più piccola unità di elaborazione deployabile) e distribuiscono le richieste di rete tra i Pod disponibili con conseguente distribuzione tra i nodi.
- *Workload Balancing o Automated scalability*: processo attraverso il quale Kubernetes, valutando il carico di lavoro, può automaticamente scalare, aumentare o ridurre, il numero di repliche disponibili per soddisfare la domanda ottimizzando l'utilizzo di risorse. Questo meccanismo garantisce che nessuna singola istanza venga sovraccaricata, migliorando la disponibilità e la reattività dell'applicazione.
- *Automated Rollout/Rollback*: rollout è il processo di distribuzione di una nuova versione di un'applicazione o servizio all'interno del cluster. Kubernetes esegue i rollout in modo incrementale, sostituendo gradualmente i vecchi Pod con quelli che eseguono la nuova versione. Questo approccio riduce al minimo i rischi di do-

³<https://www.docker.com/>

⁴<https://kubernetes.io/>

wntime, poiché se qualcosa va storto durante il processo, solo una parte limitata dell'applicazione è coinvolta.

Se durante un rollout si verifica un problema, Kubernetes può eseguire automaticamente un rollback, cioè riportare l'applicazione alla versione precedente, che è nota per funzionare correttamente.

- *Self-healing*: il sistema monitora costantemente lo stato dei container e, in caso di blocco o mancata risposta agli *health-check*, li riavvia automaticamente. Garantisce inoltre che il traffico non venga indirizzato verso container che non sono ancora pronti a gestire richieste, assicurando così un funzionamento continuo e affidabile delle applicazioni.

Esempio 2.1.1. Il funzionamento completo, unendo le due tecnologie, è strutturato come segue: comincia con la creazione di una *docker image* o la scelta di una disponibile, la quale verrà successivamente utilizzata come immagine del *Pod* di Kubernetes (la più piccola unità di elaborazione deployabile). I Pod possono essere successivamente inseriti all'interno di *workload manager* come *Deployment*, *ReplicaSet*, *DaemonSet* e *StatefulSet* per gestirne il carico di lavoro ed eseguirli in modo controllato e automatizzato, garantendo scalabilità e disponibilità tramite repliche.

Una volta creato l'applicativo è possibile esporre le sue funzionalità in due modi principali: i *Services* e gli *Ingress*; il primo espone una applicazione all'interno della rete permettendo all'utente di interagirci mentre il secondo garantisce l'accesso dall'esterno della rete ad un servizio nel cluster.

Il tutto viene racchiuso all'interno di un *Namespace*, un'entità che fornisce un meccanismo per isolare gruppi di risorse all'interno del cluster.

2.1.3 Edge Cloud Continuum

Come visto in [7] le infrastrutture informatiche, in continua evoluzione per soddisfare i sempre più stringenti requisiti, qualche decennio fa hanno visto nel *cloud computing* (data-center distribuiti su larga scala) una soluzione alle esigenze prestazionali e di scalabilità degli attuali sistemi; gli utenti accedono ai servizi e alle risorse, previo abbonamento, comunicando con il *data-center*. Questa modalità operativa implica uno svantaggio intrinseco nel suo funzionamento: la costante necessità di trasferire dati da e verso utenti e data-center provoca un aumento dei costi e della latenza. Con gli attuali progressi tecnologici si è riusciti, in parte, a disaccoppiare i data-center dall'essere l'unica risorsa di calcolo presente in un sistema, consentendo di allocare parte dello sforzo computazionale in dispositivi di calcolo più piccoli e vicini all'utente, operando quindi da punti intermedi; sono i così detti *Edge Nodes*. Questa soluzione riduce l'impatto dei problemi visti sopra, andando a spostare l'elaborazione più vicino al luogo in cui vengono generati i dati e andando a pre-elaborare o risolvere direttamente nel nodo Edge le richieste (evitando quindi di connettersi al data-center); questo a discapito di una

maggiore complessità architetturale. Questa architettura viene utilizzata in situazioni di bassa latenza che richiedono elevata disponibilità, elaborazione in tempo reale o gestione locale di informazioni riservate.

La più semplice definizione di **edge cloud** è “processo di elaborazione e archiviazione che avviene ai margini della rete” dove con margine della rete ci si riferisce agli elementi di un’infrastruttura che sono lontani dal *core*. In particolare ci si riferisce ai luoghi in cui i dati vengono generati o consumati i quali, per natura della rete, hanno una ampia distribuzione geografia. Nella Figura 2.1 viene mostrata una visione generale di un’architettura *edge-cloud*.

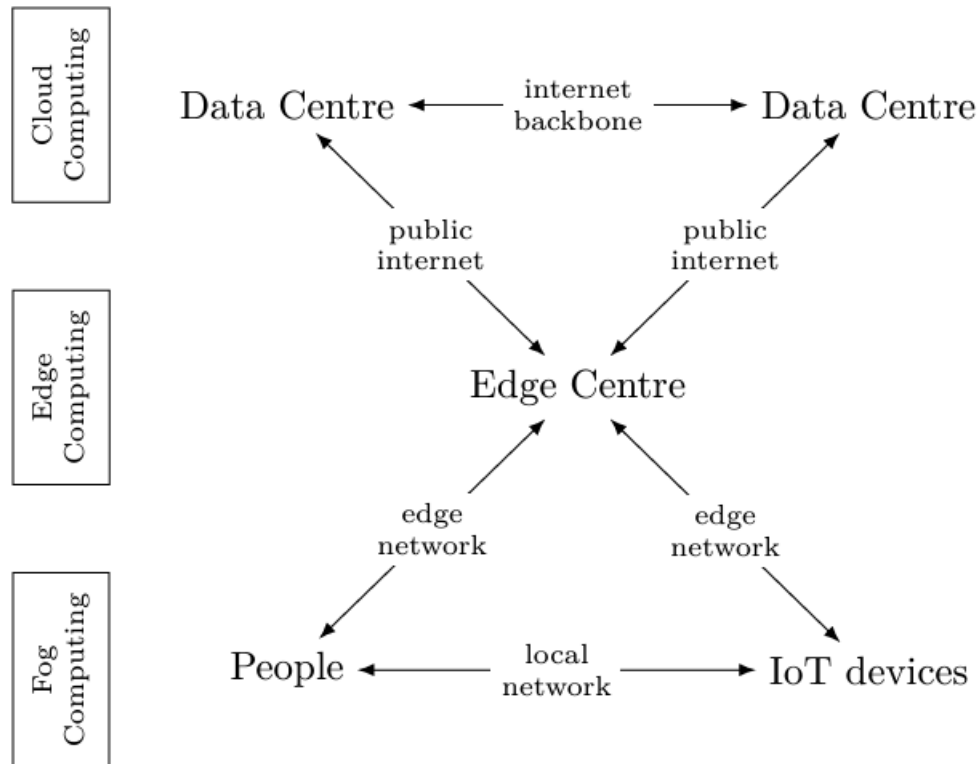


Figura 2.1: Panoramica sull’infrastruttura Edge Cloud [7]

In [8] vengono raccolti 36 studi contenenti proposte di definizione per il **Cloud Continuum** e viene presentata come definizione complessiva “*an extension of the traditional Cloud towards multiple entities (e.g., Edge, Fog, IoT) that provide analysis, processing, storage, and data generation capabilities*”. Applicando questa definizione all’*edge cloud continuum* si va a definire *edge* e *cloud* come entità presenti gestite come un’unica componente. Le maggiori problematiche che il Continuum incontra con l’integrazione di nodi Edge sono dovute alla forte distribuzione geografica a cui sono soggetti gli elementi del sistema. Le diverse distanze fisiche tra i nodi e i data-center introducono una variabilità nelle prestazioni e nella latenza a seconda della localizzazione, rendendo più difficile la ca-

pacità dei *service provider* di soddisfare determinati Service Level Agreement (SLA) [9]. I nodi edge richiedono anche opportune verifiche di proprietà di sicurezza e privacy, verificabili con maggiori sforzi, in quanto coinvolgono infrastrutture di terze parti. Sorgono inoltre difficoltà nel mantenere la coerenza tra i dati in un ambiente così distribuito, vengono infatti richiesti protocolli sofisticati di sincronizzazione tra i data-center e i nodi edge che possano gestire eventuali conflitti o incongruenze derivanti dalla latenza della rete, la quale può provocare la ricezione di dati *obsoleti* che comprometterebbero lo stato del sistema, e dall'elaborazione locale (dati elaborati localmente prima di essere sincronizzati col cloud). Ogni nodo possiede risorse computazionali e di storage differenti e limitate, questa eterogeneità fisica accentua la complessità nel sincronizzarsi in quanto i meccanismi e i protocolli da implementare dovranno adattarsi dinamicamente alle risorse dei vari nodi.

Nell'ambito delle infrastrutture distribuite moderne, l'Edge-Cloud Continuum rappresenta un paradigma emergente che unisce le capacità del cloud computing con l'elaborazione e l'archiviazione dati agli estremi della rete (*edge computing*), il tutto in un'unico sistema complessivo (*continuum*). Offre una piattaforma flessibile e scalabile che consente di distribuire le risorse computazionali tra cloud centralizzati e nodi edge situati più vicini agli utenti finali e ai dispositivi IoT, comportando una gestione di entità distribuite geograficamente nel territorio.

Un sistema distribuito così complesso necessita tuttavia di un'adeguata soluzione per il monitoring e la verifica del suo corretto funzionamento. La natura eterogenea e geograficamente distribuita di un'infrastruttura Edge-Cloud richiede strumenti che possano garantire l'ottenimento di metriche da tutti i nodi del sistema per valutarne il relativo comportamento e rilevare anomalie o guasti, ottenendo così una soluzione di monitoring efficace.

2.2 Monitoring

In un sistema distribuito, l'affidabilità e le prestazioni dipendono dalla coordinazione e dall'integrità delle varie parti che lo compongono, rendendo il monitoraggio una pratica essenziale per garantire il corretto funzionamento dell'intero sistema.

Il monitoring è quella tecnica che si concentra sulla collezione di *evidence*, o prove, per stabilire lo stato di salute e il comportamento del sistema.

Si basa su *tre* elementi fondamentali:

- **Logs:** registrazioni testuali di eventi che si verificano all'interno di un applicazione o di un sistema; possono includere errori, avvisi, informazioni operative e dati diagnostici. Sono informazioni strutturate in modo *human-readable* in quanto pensate per essere analizzate manualmente. La gestione di grandi volumi di log

provenienti da molteplici sorgenti è un problema che spesso trova soluzione nell'utilizzo di formati *machine-readable* che permettono l'automatizzazione del processo di analisi.

- **Traces:** forniscono una visione dettagliata del percorso (*trace*) che una richiesta segue in un sistema, salvando sia lo stato attuale che il relativo contesto di esecuzione. Ogni traccia è costituita da una serie di segmenti detti *span* che rappresentano singole operazioni.
- **Metrics:** dati quantitativi che descrivono lo stato e le prestazioni di un sistema nel tempo. Includono informazioni come l'utilizzo della CPU, la memoria disponibile, il numero di richieste per secondo, la latenza delle risposte etc. Fondamentali per il monitoraggio in tempo reale e l'ottimizzazione delle prestazioni, offrono una panoramica immediata dello stato di salute del sistema.

Logs, traces e metrics sono strumenti complementari nel monitoraggio dei sistemi. Mentre i *logs* forniscono un registro dettagliato degli eventi, le *traces* permettono il tracciamento delle operazioni legate ad uno specifico evento a vari livelli e le *metrics* consentono una misura oggettiva dello stato interno di un sistema, misure alla base della verifica di *PNF*, approfondite nella Sezione 2.3.

I primi sistemi di monitoring realizzati basati su metriche erano *collettori di metriche* *Simple Network Management Protocol (SNMP)* il cui funzionamento è basato su un'applicazione centrale che interroga gli agenti SNMP installati in ogni dispositivo per ottenere i dati raccolti e successivamente analizzarli. È un modello di monitoring completamente centralizzato e ormai superato. La realizzazione di un sistema di monitoring attuale, invece, pone di fronte a varie scelte in merito all'architettura che il sistema andrà ad implementare, ciascuna caratterizzata da specifiche peculiarità. Alcuni esempi di scelte progettuali riguardano sia l'*architettura* che l'applicativo di monitoring andrà ad avere, che l'effettivo procedimento di *estrazione delle informazioni*.

2.2.1 Architettura di monitoring e relative applicazioni

Vengono analizzati alcuni esempi di tipiche architetture presenti nel mercato.

- **Sistemi Completamente Centralizzati:** sia la raccolta dei dati che il loro storage avvengono in un singolo punto centralizzato dove vengono in seguito processati e analizzati. Questo approccio fornisce semplicità, consistenza e affidabilità. Fornisce anche maggiore sicurezza e privacy in quanto i dati sono custoditi e processati in un unico ambiente. Dall'altro lato questo approccio causa alcuni svantaggi come un elevato sovraccarico del traffico di rete verso il nodo centrale e una limitata

scalabilità. Graylog⁵ adotta questa architettura.

- **Sistemi Distribuiti:** permettono di monitorare consistentemente i dispositivi nella rete raccogliendone le relative metriche. La raccolta ed elaborazione distribuita dei dati può ridurre la quantità di dati che devono essere trasmessi, il che può far risparmiare risorse di rete e migliorare l'efficienza. Viene inoltre migliorata la scalabilità.
 - **Raccolta Distribuita ma Storage Centralizzato:** i dati vengono raccolti da vari agenti o nodi distribuiti, ma vengono inviati a un punto centrale per lo storage e l'analisi. Alcuni servizi che si basano su questa architettura sono Prometheus⁶ e Logstash⁷.
 - **Completamente Distribuito:** sia la raccolta che lo storage dei dati sono distribuiti tra più nodi. I dati vengono memorizzati e processati in vari punti della rete, spesso con meccanismi di replica e bilanciamento del carico. Alcuni servizi che si basano su questa architettura sono Elasticsearch⁸ e Kafka⁹.

2.2.2 Estrazione informazioni

Un aspetto cruciale della progettazione è rappresentato dalle modalità con cui i dati o informazioni vengono estratti dai sistemi monitorati e trasferiti al sistema di raccolta e analisi. Le due modalità principali che permettono al sistema di monitoring di ottenere questi dati sono il *pull* e il *push*.

- **Pull:** in questa modalità il sistema di monitoring richiede attivamente (*polling*) i dati ai sistemi che sta monitorando; le metriche possono essere esposte sia dall'applicazione stessa (*monitoring nativo*) che da *agenti*, i quali si occupano di ottenere preventivamente le misurazioni. È a tutti gli effetti una richiesta esplicita da parte del monitor verso gli *endpoint* del sistema target. Un'esempio applicativo che segue questo paradigma è Prometheus⁶
- **Push:** in questa modalità il sistema di monitoring si aspetta di ricevere i dati su un interfaccia configurata per poterli elaborare; è il sistema monitorato ad inviare attivamente informazioni. Le metriche possono arrivare sia direttamente dall'applicazione che da agenti esterni. Un'esempio di applicativi che seguono questo paradigma sono Graylog⁵ e Elasticsearch⁸

⁵<https://graylog.org/>

⁶<https://prometheus.io/>

⁷<https://www.elastic.co/logstash>

⁸<https://www.elastic.co/elasticsearch>

⁹<https://kafka.apache.org/>

Come accennato nell'elenco, un altro aspetto legato al reperimento delle metriche riguarda la distinzione tra *monitoring nativo*, detto anche *agent-less*, e monitoring tramite *agenti esterni*, detto *agent-based*.

- **Agent-based:** è una tecnica di monitoraggio che richiede l'installazione di un software, chiamato *agente*, su ogni dispositivo da monitorare il quale è in grado di raccogliere e collezionare dati. Come descritto prima può essere configurato per interagire con un sistema che lavora in pull o push, rispettivamente risponde alle richieste che gli vengono fatte oppure invia autonomamente i dati. Questa metodologia, con gli adeguati permessi concessi all'agente, fornisce informazioni dettagliate sul sistema ma, dall'altro lato, utilizza un numero di risorse notevole nella macchina host e fornisce un possibile vettore di attacco aggiuntivo. Fornisce anche resilienza a problemi di rete in quanto un agente salva localmente le metriche rilevate.
- **Monitoring nativo o trasparente:** in questa tecnica non è necessaria l'installazione di nessun client per la raccolta dei dati sul dispositivo da monitorare ma si utilizzano strumenti e tecnologie di monitoraggio direttamente forniti all'interno delle applicazioni, evitando installazioni e configurazioni aggiuntive. L'applicazione autonomamente raccoglie le proprie metriche e le espone secondo una delle modalità precedentemente elencate: può inviarle periodicamente (*push*) oppure attendere di essere interrogata (*pull*). È più semplice da configurare e comporta un impatto significativamente inferiore sulle risorse del sistema host rispetto alla controparte con agenti; richiede tuttavia l'apertura di porte specifiche e dipende fortemente dalla qualità della connessione per effettuare il polling attivo dei dati.

Formati e protocolli standard

L'obiettivo di ricercatori e sviluppatori è quello di utilizzare *formati e protocolli standard* sia per lo scambio di dati che per la comunicazione. Vengono garantite importanti proprietà come l'*interoperabilità* tra diversi sistemi, prodotti e servizi permettendone la cooperazione senza problemi; *sicurezza*, *qualità* e *performance* sono dovuti alle innumerevoli ricerche e utilizzi a cui una tecnologia viene sottoposta prima di essere definita standard. Altri pregi dovuti all'uso di standard sono la riduzione dei costi, sia di sviluppo che tecnologici, e la conformità legislativa ovvero il rispetto di *standard legislativi* imposti.

Per integrare applicazioni e sistemi di monitoring in modo efficiente, si fa dunque largo uso di formati e protocolli standard. L'adozione di tecnologie consolidate, come l'uso di API REST con body in formato JSON o di protocolli di rete standard come SNMP o *Syslog*, utilizzato per trasmettere e raccogliere semplici informazioni di logging dai diversi dispositivi della rete, facilita l'interoperabilità tra le applicazioni e i sistemi di monito-

ring, rendendo possibile l'invio e la ricezione di dati in modo standardizzato e leggibile. Sono presenti anche standard imposti da applicativi o framework di monitoring *open-source* come Prometheus⁶ o OpenTelemetry¹⁰ con il relativo OpenTelemetry Protocol (OTLP) che rispettivamente trattano i dati come una time-series (flusso di valori con timestamp appartenenti alla stessa metrica) o come una struttura dati da loro definita (OpenTelemetry Model) (open-telemetry utilizza exporter che inviano i dati in formato OTLP, essendo uno standard viene supportato da vari servizi di monitoring come Prometheus, Jaeger, ecc). Sono presenti anche standard per la trasmissione di metriche in ambienti *cloud-native* come OpenMetrics¹¹. Questa standardizzazione non solo semplifica l'integrazione e riduce la complessità di sviluppo, ma assicura anche una maggiore compatibilità tra diversi ambienti e piattaforme, migliorando l'efficienza operativa e la scalabilità del sistema.

2.3 Proprietà Non Funzionali

Per far fronte alle stringenti necessità di ottenere una visione sempre più completa sullo stato di un sistema sono state introdotte, al fianco delle proprietà funzionali, le **PNF** il cui scopo è quello di definire *come* il sistema svolge le sue funzioni concentrandosi su aspetti di *qualità*, a differenza delle proprietà funzionali che definiscono *cosa* effettivamente viene fornito. Una PNF di un sistema è un vincolo al modo in cui il sistema implementa e fornisce le sue funzionalità. In altre pubblicazioni presenti in letteratura viene usato il termine *Non Functional Requirements* riferendosi a PNF con annesso il criterio di accettazione (requisito) da soddisfare; la metodologia adottata in questa tesi esprime formalmente i Non Functional Requirements e la loro verifica tramite Contratti, illustrati nella successiva Sezione 2.4.1.

In questa pubblicazione [10] vengono analizzati 36 paper aventi come topic le architetture a microservizi, tra questi 21 discutono le tecniche per verificare *Non Functional Requirements*. La proprietà più discussa in letteratura risulta essere l'*affidabilità*, seguita da *scalabilità* e *disponibilità*.

Di seguito vengono elencati e illustrati alcuni esempi di *PNF* che possono essere misurate e valutate in un sistema.

- **Reliability:** affidabilità, capacità di funzionare correttamente e senza interruzioni per un determinato periodo di tempo, garantendo prestazioni stabili.
Definizione ISO [11] "*Degree to which a system, product or component performs specified functions under specified conditions for a specified period of time*".

¹⁰<https://opentelemetry.io/>

¹¹<https://openmetrics.io/>

- **Performance:** prestazioni, capacità di un sistema di eseguire le sue funzionalità in modo efficiente rispettando dei requisiti e aspettative definiti dall'utente.
Definizione ISO [11] *"the degree to which a product performs its functions within specified time and throughput parameters and is efficient in the use of resources (such as CPU, memory, storage, network devices, energy, materials...) under specified conditions"*.
- **Scalability:** scalabilità, capacità di un sistema di gestire un aumento del carico di lavoro senza compromettere prestazioni o affidabilità.
Definizione ISO [11] *"Degree to which a product can handle growing or shrinking workloads or to adapt its capacity to handle variability"*.
- **CIA - Confidentiality:** confidenzialità, garantisce che i dati vengano acceduti solo da chi effettivamente autorizzato e non vengano divulgati.
Definizione ISO [11] *"Degree to which a product or system ensures that data are accessible only to those authorized to have access"*.
- **CIA - Integrity:** integrità, garantisce che i dati siano accurati, completi e non alterati in modo non autorizzato; in altre parole i dati sono corretti e attendibili.
Definizione ISO [11] *"Degree to which a system, product or component ensures that the state of its system and data are protected from unauthorized modification or deletion either by malicious action or computer error"*.
- **CIA - Availability:** disponibilità, il sistema è accessibile e operativo quando richiesto.
Definizione ISO [11] *"Degree to which a system, product or component is operational and accessible when required for use"*.
- **Access control:** controllo accessi, solo gli utenti autorizzati dalle politiche del sistema possono accedere alle risorse
- **Resilience:** resilienza, capacità di un sistema di continuare a funzionare correttamente, e di recuperare rapidamente, in seguito ad un guasto.
Definizione ISO [11] *"Degree to which a product can automatically place itself in a safe operating mode, or to revert to a safe condition in the event of a failure"*.

È importante sottolineare il fatto che queste proprietà definiscono degli obbiettivi concettuali da raggiungere ma non trattano le modalità con cui si verifica l'effettivo soddisfacimento. Questo accade perché le modalità di verifica differiscono in base all'entità che si sta controllando ed è compito di un *esperto del sistema*, colui che ha adeguate conoscenze e competenze, andare a definire come una proprietà si può concretizzare in uno specifico contesto. Gli esempi illustreranno come l'espressione pratica dello stesso concetto è strettamente legata al contesto di applicazione.

Esempio 2.3.1. Valutando l'*availability* di un server si misura il tempo durante il quale il server è *up* e funziona correttamente (riceve le richieste e risponde fornendo il relativo servizio) mentre valutando l'*availability* di una rete cellulare si va a misurare la capacità della rete di fornire connessione ai dispositivi mobili in una determinata area geografica.

Esempio 2.3.2. Valutando l'*Integrity* di una comunicazione di rete ci si riferisce alla capacità di trasmettere e ricevere i dati senza alterazioni o perdite mentre valutando l'*Integrity* di uno *storage* (come un database o un dispositivo di archiviazione fisico) ci si riferisce alla capacità di garantire che i dati archiviati non vengano alterati e rimangano completi.

2.4 Verifica di Proprietà non Funzionali - Assurance

Le tecniche di **assurance** mirano a garantire che un sistema *verifichi*, e quindi soddisfi, determinate Proprietà Non Funzionali precedentemente definite.

Per questa tesi viene adottata la **metodologia di assurance** utilizzata in [7], e riportata in questa sezione, che si occupa di misurare e valutare PNF di un'infrastruttura target sulla base delle *evidence* raccolte dagli endpoint di monitoring. La verifica viene formalizzata attraverso l'uso di contratti basati sulle *evidence* raccolte. Questa metodologia adottata per il processo di *assurance* mira a valutare un sistema indipendentemente dallo stato in cui si trova durante la valutazione (valutazione statica) basando i suoi risultati su misurazioni raccolte, archiviate in uno storage e indicizzate nel tempo invece che su misure dirette, permettendo così la definizione di contratti basati su intervalli temporali.

Ogni componente del sistema possiede uno *stato interno*, rappresentazione dinamica dello stato delle variabili e delle risorse, e una *configurazione*, configurazione statica del componente che ne caratterizza il comportamento, utilizzati per valutarne determinate proprietà. Questi due dati (stato interno e configurazione) vengono esposti dal componente del sistema al processo di assurance tramite interfacce di monitoring standard dette anche *endpoint di monitoring*. Gli endpoint andranno a rappresentare il target del processo di assurance per la raccolta di *measurements* dall'infrastruttura.

2.4.1 Assurance methodology

La Figura 2.2 mostra lo schema della metodologia di assurance che verrà utilizzato nel corso di questa tesi.

La metodologia adottata in [7] utilizza gli endpoint delle infrastrutture come fonte di misurazioni (espongono lo stato e le configurazioni del sistema). Le misurazioni degli

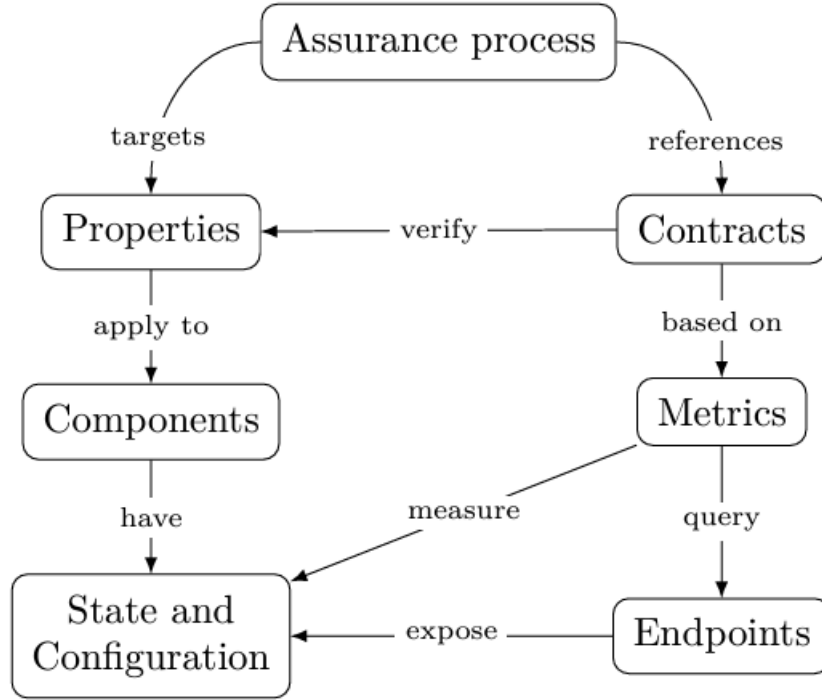


Figura 2.2: Schema di *assurance methodology* proposto in [7]

endpoint vengono valutate attraverso **metriche** le quali si focalizzano sulla valutazione di un comportamento specifico per verificare PNF. Le metriche, una volta valutate e ottenute le relative *evidence*, diventano la base per la valutazione dei **contratti**, i quali fanno riferimento direttamente a una PNF specifica e descrivono come può essere verificata in modo formale. Individuiamo nelle *metriche* e nei relativi *contratti* gli elementi focali di questa sezione.

Metriche Una metrica è una funzione associata da una funzionalità, la quale vuole misurare i suoi aspetti o comportamenti sui componenti che la implementano; è un *qualsiasi tipo di trasformazione che viene applicato ai dati* per trasformarli in una forma più utile. La più semplice trasformazione, definita identità, consiste nel tenere i dati grezzi “così come sono”. Esistono poi trasformazioni che vanno a ridurre la quantità di dati raccolti rispetto al totale reperibile dal sistema, mirando ad estrarre proprietà o informazioni specifiche da essi. Vengono definite sugli stati e sulle configurazioni esposte dagli endpoint di monitoring. La valutazione delle metriche fornisce le *evidence*, o prove, misurazione oggettive atte a confermare se una particolare proprietà è valida.

Contratti Descrivono formalmente come validare le proprietà di un servizio. Un contratto è una funzione booleana il cui scopo è quello di valutare la validità o meno di una

determinata proprietà. La valutazione avviene basandosi sulle *evidence*, o prove, ottenute tramite valutazione delle metriche degli endpoint. Un contratto è una tupla costituita dal *istante di tempo* in cui è avvenuta la valutazione e dal *set di componenti* che implementano il servizio target. Un aspetto cruciale dei contratti è essere *time-dependant*, l'esito dipende quindi dall'istante temporale in cui è stato valutato il contratto. Per garantire consistenza, questa metodologia mira a definire un solo contratto per ogni proprietà (mapping bigettivo tra proprietà e contratto).

2.4.2 Assurance process

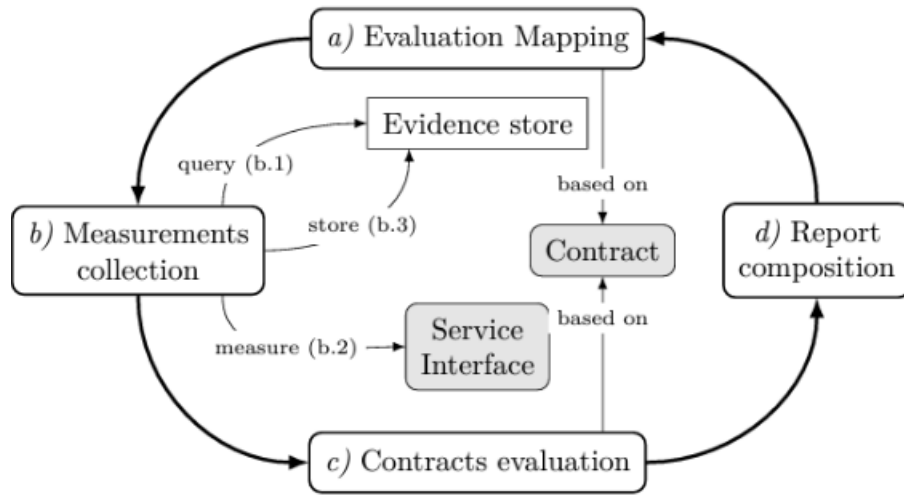


Figura 2.3: Schema di *assurance process* proposto in [7]

Un processo di assurance dipende da una *assurance methodology* per validare le proprietà. La soluzione adottata in [7] supporta la verifica continua e collaborativa di PNF delle componenti infrastrutturali. La Figura 2.3 applica l'*assurance methodology* vista nella Sezione 2.4.1 suddividendo il processo in 4 step ripetuti ciclicamente.

- (a) *Evaluation mapping*: nel primo step, partendo dalle *proprietà* che si intendono verificare si va ad identificare le *metriche* e i *contratti* che dovranno essere valutati per poter effettuare quegli accertamenti
- (b) **Measurement collection**: il passo successivo consiste nella valutazione delle metriche (*metrics evaluation*) precedentemente identificate e fornite dal servizio, step b.2, e nella seguente raccolta e immagazzinamento delle *evidence* generate (*measurements*), step b.3.
- (c) **Contracts evaluation**: nel terzo passaggio viene eseguita l'effettiva verifica delle proprietà, calcolando il risultato del *contratto*, utilizzando le misurazioni raccolte

in precedenza (*measurements* usate come *evidence* per comprovare un contratto) e restituendo un output booleano.

- (d) *Report composition*: nell'ultimo passaggio i risultati parziali, *measurements* e *contracts evaluation*, vengono aggregati in un report che include una descrizione dei risultati (proprietà verificate o no) e le relative *evidence* a supporto. Il report viene generato seguendo un formato *machine readable* per garantirne il facile utilizzo in altri processi automatizzati, come la certificazione descritta nella sezione successiva.

2.5 Certificazione di Proprietà non Funzionali - Certification

La **certificazione** è una tecnica di *security assurance* il cui scopo è dimostrare, seguendo uno *schema di certificazione*, che un sistema target, in un determinato istante di tempo, si comporti come previsto e soddisfi determinate PNF. Il funzionamento è costituito in parte dalla *verifica di PNF* precedentemente illustrata nella Sezione 2.4 ma con l'aggiunta di una Certification Authority (CA) il cui compito è la gestione sicura del processo di verifica garantendo sicurezza, affidabilità e ripetibilità. La certificazione, come menzionato in [12], è stata integrata nel ciclo di vita dei sistemi distribuiti, partendo dallo sviluppo degli stessi, per aumentare la qualità del servizio e garantire un comportamento stabile nel tempo.

Per questa tesi viene adottata la metodologia menzionata in Ardagna et al. [13] utilizzando la loro nomenclatura. La Figura 2.4 mostra un generico schema di certificazione contenente le entità coinvolte (CA, Accredited Lab (AL) e Target) e le varie interazioni per giungere ad un certificato. È fondamentale la creazione di una *chain-of-trust* tra l'*utente* che chiede la certificazione, la CA che prepara il *certification model*, l'AL che esegue il *certification model* e il certificato rilasciato. Segue un'illustrazione delle componenti presenti nello schema.

Target rappresenta l'obiettivo del processo di certificazione ed è costituito dalle implementazioni concrete dei componenti del sistema a cui il certificato è rivolto (un software, un servizio o un sistema in senso più ampio).

CA Ha il compito di definire e orientare tutte le attività necessarie per valutare e certificare una specifica proprietà su un Target; lo fa definendo un *Certification Model* che contiene un *Evidence Collection Model*. Fornisce delle linee guida fondamentali per

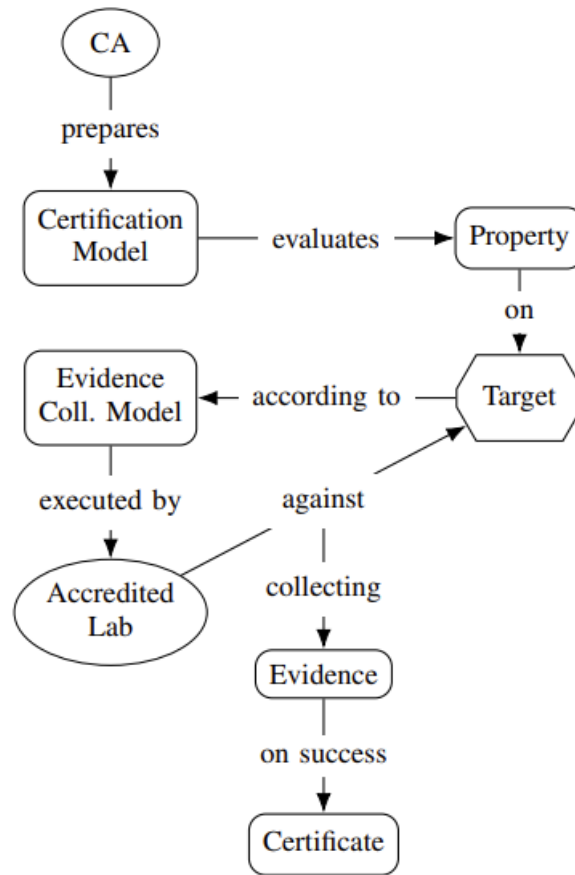


Figura 2.4: Generico processo di certificazione [13]

assicurare che il processo di certificazione sia condotto in modo sistematico e conforme ai requisiti predefiniti garantendo sicurezza e affidabilità.

Certification Model Una specifica delle attività da eseguire per certificare PNF sul sistema target. Il modo più conciso di rappresentarlo è una tupla costituita da *Proprietà* che si intendono verificare, *Target* ed *Evidence Collection Model*. È parte fondamentale di un certificato in quanto definisce cosa si sta certificando (proprietà), come (*evidence collection model*) e chi si sta certificando (target) andando a definire concretamente come deve avvenire la certificazione.

Evidence tutti i dati raccolti e valutati in modo oggettivo su un sistema target, include sia i dati delle metriche che le valutazioni fatte

Evidence Collection Model coordina le attività necessarie a raccogliere *evidence* dal sistema target, le quali verranno valutate per stabilire il rilascio del certificato o meno. Definisce come i dati raccolti dal sistema di monitoring vanno all'AL.

AL sono laboratori riconosciuti ufficialmente da un ente di accertamento e richiedono di rispettare determinate norme (ad esempio [14]: *General requirements for the competence of testing and calibration laboratories*) per garantire affidabilità e precisione dei risultati. Viene delegato dalla CA per eseguire l'*Evidence Collection Model* sul sistema target; con i dati raccolti, le *evidence*, il laboratorio potrà assegnare un certificato al sistema in caso di valutazione positiva

Certificato il certificato viene rilasciato dalla CA ai sistemi target che superano con successo la valutazione di determinate PNF sulla base delle *evidence* raccolte, ne attesta la conformità. Contiene un riferimento al *certification model*, il quale ne illustra i dettagli, e alle *evidence* raccolte che ne supportano la validità.

Flusso di esecuzione: La CA definisce il *Certification Model* costituito dalle proprietà che devono essere verificate su un sistema target seguendo uno specifico *Evidence Collection Model*. L'AL, delegato dalla CA, ha l'incarico di raccogliere le prove o *evidence* dal sistema. Se le *evidence* dimostrano positivamente la conformità della proprietà viene rilasciato un *certificato* che attesta la conformità del sistema ai requisiti stabiliti.

2.6 Related Work

In [15] viene proposta una metodologia di valutazione multi-layer (monitora il livello fisico, le VM e le applicazioni in esecuzione sulle VM raccogliendo metriche a vari livelli di granularità) e continua, fondata sui Common Criteria (CC)¹², per valutare i livelli di sicurezza delle applicazioni Cloud, in termini di Assurance Level, attraverso l'aggregazione delle proprietà di sicurezza, raccolte su vari livelli, dei singoli componenti che le costituiscono. Fornisce al proprietario del servizio una panoramica della sicurezza del suo servizio senza la necessità di analisi manuali dettagliate.

In [16] viene proposto un framework di certificazione della sicurezza test-based (test-based assurance: le evidence arrivano da test e non da strumenti di monitoring) per servizi cloud. Fornisce una metodologia e un ambiente di cloud-engineering basati su certificati con il relativo set di componenti che gestisce il processo di certificazione, online e semi-automatico, di servizi/applicazioni in un sistema cloud-based; fornisce

¹²<https://www.commoncriteriaportal.org/>

anche dei tools in supporto alla chain-of-trust imposta dalla certificazione. Il framework infine supporta la valutazione di sistemi cloud sia pre-deployment che in production-deployment a diversi livelli, discostandosi dai tradizionali strumenti che non considerano il processo di sviluppo.

In [17] viene proposta una rigorosa e adattiva tecnica di assurance basata sulla certificazione che mira ad un ecosistema cloud trasparente ed affidabile dove ogni parte del cloud si comporta come previsto. Utilizza uno schema di certificazione basato su test per dimostrare il soddisfacimento di PNF dei servizi cloud; una CA, online e disponibile durante l'intero processo, definisce i requisiti non funzionali da verificare su un determinato modello. Il controllo avviene in modo semi-automatico tramite un algoritmo di matching che effettua un confronto tra il certification model definito dalla CA e un'istanza. Presenta inoltre un processo continuo e incrementale di gestione del ciclo di vita di un certificato (semi-)automatico, dall'emissione al suo riadattamento a cambi contestuali (cloud multilayer dinamici) alla revoca. Il framework è stato applicato in uno scenario industriale reale per la gestione di un sistema di pagamento (ENGpay) e ne sono state valutate prestazioni, qualità, applicabilità e usabilità pratica.

In [18] viene proposto un framework di gestione del rischio per sistemi distribuiti basato su tecniche di *assurance*, le quali monitorano il comportamento del sistema tramite valutazione di PNF, che soddisfa i requisiti di *gestione del rischio* imposti dai moderni sistemi distribuiti; statistiche come performance e qualità del framework sono state misurate in uno scenario simulato di industria 4.0. Questo framework fornisce un'espansione ai framework di risk-management consentendo alle organizzazioni grandi e complesse di valutare il proprio rischio basandosi su risultati derivanti dalla valutazione di PNF relativi all'efficacia dei meccanismi implementati, integrando quindi completamente il concetto di *assurance* alla gestione del rischio.

In questi due paper vengono proposti studi in merito all'**assurance in dispositivi IoT** le cui ricerche rispetto a servizi e sistemi cloud sono ancora agli inizi.

In [19] viene proposto un framework altamente automatizzato di security assurance per sistemi Edge e IoT, paradigmi di elaborazioni emersi negli ultimi anni, indipendente dalle complessità infrastrutturali e basato su un'architettura di Kubernetes avanzata in grado di gestire applicazioni 5G-native. Implementa un processo di assurance altamente scalabile che garantisce la verifica di PNF raccogliendo evidence dal sistema target. Come caso reale di applicazione viene presentato l'Internet of Medical Things (IoMT) considerano come contesto di esecuzione un moderno ospedale dotato di molti device eterogenei (sensori umidità e temperatura, dispositivi IoT medici ecc).

L'anno successivo in [20] il framework proposto in precedenza viene ampliato introducendo il concetto di assurance IoT guidata da una visione olistica del sistema di destinazione. Questo metodo definisce ed esegue valutazioni di assurance che si allineano e sfruttano l'architettura del sistema target, distribuendo e aggregando i calcoli di conseguenza. Il framework considera l'intero ciclo di vita del sistema e produce risultati a diversa granularità attraverso una valutazione di assurance basata su attività multi-layer, multifase

(in diversi momenti del suo ciclo di vita) e comportamentali. Il framework proposto è stato valutato in uno *Smart Light System* reale.

Come visto in [13], la certificazione di moderni sistemi distribuiti incontra alcune *difficoltà* dovute sia al continuo adattamento ed evoluzione dei sistemi, implicando l'invalidamento dei corrispondenti certificati (la cui realizzazione richiede sforzo in termini di tempo e risorse), che alla presenza di Machine Learning (ML) a livello applicativo e infrastrutturale. Queste caratteristiche rendono il comportamento del sistema imprevedibile, in contrasto con i principi fondanti della certificazione quali *staticità*, *predicibilità* e *verificabilità*.

Lo schema di certificazione proposto in [21] va oltre la semplice valutazione del comportamento di un servizio, prendendo in considerazione fattori aggiuntivi che descrivono, ad esempio, come il servizio è stato implementato e verificato, con l'obiettivo di aumentare la qualità e le prestazioni di un sistema distribuito. Questo approccio è definito certificazione multidimensionale, poiché include dimensioni aggiuntive che modellano aspetti rilevanti (ad esempio, linguaggi di programmazione e processi di sviluppo) che contribuiscono in modo significativo alla qualità dei risultati della certificazione; l'obiettivo non si limita più alla valutazione delle PNF degli artefatti software ma viene esteso per considerare *dimensioni* aggiuntive, come ad esempio i processi di sviluppo e verifica. Questo schema certifica le proprietà di un dato servizio considerando ogni dimensione in modo indipendente: compone i risultati della valutazione in ogni dimensione e produce un certificato accurato del comportamento del servizio. Lo schema infine è integrato nella gestione del ciclo di vita (dallo sviluppo del sistema al suo funzionamento e alla sua manutenzione) di sistemi distribuiti per supportare una selezione dei servizi basata sulla certificazione, in cui i servizi funzionalmente equivalenti vengono classificati (service ranking) in base alle PNF certificate che detengono e ai requisiti degli utenti corrispondenti, utilizzando una tecnica di Multi-Criteria Decision-Making (MCDM). Questa modellazione porta a certificati più accurati e, di conseguenza, a un processo decisionale più accurato quando i servizi/software vengono selezionati dinamicamente in fase di esecuzione sulla base dei loro certificati.

In conclusione questo approccio offre vantaggi per tutte le parti coinvolte: i fornitori di servizi ottengono certificati che riflettono meglio tutte le best practice seguite; gli utenti finali possono fare scelte più consapevoli grazie a certificati dettagliati e ben strutturati; la CA fornisce uno schema di certificazione più utile.

Capitolo 3

Implementazione

In questo capitolo viene descritto il processo di implementazione della soluzione proposta per la valutazione di sistemi distribuiti. L'obiettivo principale è illustrare i dettagli dell'infrastruttura creata, i componenti utilizzati e il funzionamento generale del sistema. La prima parte si concentra sul *target* dell'assurance, ovvero l'ambito dei servizi e delle applicazioni monitorati. Successivamente, si descrive l'*infrastruttura sperimentale* utilizzata per il deployment basata su MicroK8s. La parte centrale del capitolo si focalizza sul deployment e sul funzionamento dello *stack* fornito da *Elasticsearch*. Infine si tratta l'*Assurance Engine*, ovvero il framework di monitoring in corso di sviluppo.

3.1 Assurance Target

Il sistema implementato ha come *scope*, ovvero obbiettivo di monitoring, una determinata tipologia di servizi, concentrandosi su applicazioni o Virtual Machine (VM) containerizzate e deployate all'interno del cluster Kubernetes del laboratorio. In altre parole, il target del sistema sono esclusivamente entità (applicazioni o VM) distribuite internamente al cluster, che possono presentare una struttura centralizzata o distribuita. Le entità classificabili come possibili esempi di target vanno dai generici servizio deployabili in un cluster kubernetes come un web server o un database ad entità più specifiche come il servizio di GitLab¹ fino ad arrivare a sistemi operativi completamente deployati (VM) sul cluster tramite l'utilizzo di KubeVirt².

Una delle caratteristiche più rilevanti di questo sistema è la capacità di monitorare sia entità predisposte al monitoring trasparente, sia entità che richiedono l'ausilio di agenti per il monitoraggio, come descritto nella Sezione 2.2.2. La flessibilità di questo approccio permette di superare un'importante limitazione del monitoring, consentendo la raccolta

¹<https://about.gitlab.com/>

²<https://kubevirt.io/>

di dati da entità distribuite eseguite in container, pur non avendo un controllo diretto sul loro funzionamento in quanto si tratta di servizi o applicazioni prodotti da enti terze, “pronti all’uso” e con un’interfaccia di tipo *black-box*. Il sistema è quindi in grado di monitorare ogni tipo di applicativo distribuito all’interno del cluster. Questa integrazione è ulteriormente facilitata dalle *integrazioni* native che *Elasticsearch* mette a disposizione per interfacciarsi con vari servizi o applicativi standard, come Kubernetes o servizi offerti da Apache e Amazon Web Services (AWS), ecc.

3.2 Experimental setup

In questa sezione verrà descritto l’ambiente Kubernetes utilizzato per l’implementazione e lo sviluppo del sistema, con un’analisi delle sue configurazioni e componenti principali.

Il cluster è stato sviluppato tramite **MicroK8s**³, una distribuzione *lightweight* di Kubernetes sviluppata da Canonical⁴ e progettata per essere facilmente installabile ed eseguibile. MicroK8s è caratterizzato da un’architettura modulare che consente agli utenti di attivare solo i componenti necessari per il proprio ambiente. L’installazione predefinita include un cluster Kubernetes completamente funzionante, con l’opzione di aggiungere delle componenti (add-ons) come DNS, ingress, dashboard, e altri servizi comuni tramite semplici comandi. Fornisce quindi un cluster Kubernetes facilmente componibile. Il pacchetto include anche il supporto per il deployment di cluster distribuiti, noti anche come cluster multi-nodo.

L’infrastruttura sperimentale è quindi costituita da un cluster Kubernetes deployato tramite MicroK8s su 3 nodi fisici eterogenei presenti nel laboratorio, ognuno dei quali possiede differenti capacità in termini computazionali e di storage. Nella Tabella 3.1 vengono indicate le specifiche di ogni nodo presente, in questo modo l’architettura del sistema diventa replicabile permettendo l’esecuzione di esperimenti su un setup identico.

Tabella 3.1: Risorse disponibili nel sistema

Nodo	CPU		RAM	Disco	
	Modello	Core			Frequenza
balthasar	Broadwell	16	2.10 GHz	15.6 GiB	296.8 GiB
casper	Broadwell	16	2.10 GHz	23.4 GiB	296.8 GiB
melchior	EPYC 7453	28	2.75 GHz	252 GiB	1.7 TiB
		28	2.75 GHz		
Totale		88		291 GiB	2.3 TiB

³<https://microk8s.io/>

⁴<https://canonical.com/>

Per la gestione dello **storage distribuito**, il cluster utilizza Longhorn⁵, un sistema di storage a blocchi distribuito (*distributed block storage system*) e *open-source*. Fornisce *volumi persistenti* altamente disponibili e affidabili attraverso la loro *replica* archiviata su più nodi, assicurando così la continuità del servizio anche in caso di failure hardware o di rete (no *single-point-of-failure*). Grazie alla sua architettura leggera e all'integrazione nativa con Kubernetes, Longhorn semplifica la gestione dello storage dinamico e scalabile, supportando operazioni come il provisioning automatico dei volumi. Una delle caratteristiche chiave offerte da Longhorn sono le funzionalità di *backup* e *snapshot incrementali*, facilmente gestibili attraverso la sua interfaccia utente. Le snapshot consentono di catturare lo stato dei volumi in determinati momenti nel tempo, facilitando il ripristino rapido dei dati in caso di errori o corruzione (system side) o per la creazione di repliche. I backup, invece, permettono di salvare copie dei dati su storage esterni al cluster basandosi sugli snapshot, aumentando ulteriormente la sicurezza e la durabilità delle informazioni critiche (user side). Queste funzionalità contribuiscono a garantire l'integrità e la disponibilità dei dati del *persistent volume storage* sia all'interno che all'esterno del cluster Kubernetes, supportando strategie di disaster recovery efficaci e riducendo il rischio di perdita di dati.

Il cluster utilizza anche un plugin **Container Network Interface (CNI)**, che si occupa di configurare dinamicamente le risorse di rete e rimuoverle dai container terminati, per gestire il networking interno. Il plugin installato è Kube-ovn⁶, una soluzione avanzata e performante che combina le capacità di Kubernetes con le ricche e comprovate funzionalità di virtualizzazione di rete basate su Open Virtual Network (OVN)⁷, una piattaforma per il Software Defined Network (SDN), in un ambiente *cloud-native*; consente la creazione di reti logiche e virtuali che si sovrappongono all'infrastruttura fisica esistente. OVN si costituisce come una serie di *daemon* per *Open vSwitch*⁸ che traducono le configurazioni di rete virtuale in *OpenFlow*, un protocollo di comunicazione che permette la gestione del flusso di dati all'interno di switch e router in reti software-defined. Fornisce quindi un maggiore controllo sullo switching/routing dei pacchetti nella rete e sul binding delle porte logiche. Consente inoltre la specifica di policy di sicurezza avanzate (Firewall) e un controllo completo sulla rete (*load-balancing*, *Quality of Service*, *monitoring*). L'approccio SDN consente una gestione delle reti che separa il *control plane* della rete, la logica che decide la topologia di rete e le informazioni nelle routing tables, dal *data-plane*, che si occupa dell'effettivo inoltro del traffico, consentendo una gestione programmabile e centralizzata della rete, la quale diventa più flessibile e dinamica adattandosi alle esigenze.

Il cluster di riferimento gestisce tutte le entità, siano esse servizi o VM, come risorse di calcolo e storage creando uno strato di astrazione al di sopra di esse che consente di generalizzarne l'aspetto. Questa infrastruttura distribuita su vari nodi e con componenti

⁵<https://longhorn.io/>

⁶<https://www.kube-ovn.io/>

⁷<https://www.ovn.org/>

⁸<https://www.openvswitch.org/>

eterogenee rappresenta uno scenario ideale per **test realistici**, grazie alla sua capacità di rispecchiare ambienti applicativi complessi e variabili.

3.3 ECK Stack

La prima fase dell'implementazione è costituita dal deployment dello stack Elastic Cloud on Kubernetes (ECK)⁹ che estende le funzionalità di orchestrazione di base di Kubernetes per supportare la configurazione e la gestione di Elasticsearch, Kibana, Elastic Agent, Fleet Server, ecc. Segue un'illustrazione delle varie componenti deployate.

Elasticsearch il cuore dello stack Elastic, è un archivio dati scalabile e distribuito con implementato un motore di ricerca e analisi utilizzabile tramite query presso le API RESTful che espone. La sua architettura distribuita permette di scalare orizzontalmente, distribuendo il carico di lavoro su più nodi, garantendo così elevata disponibilità e prestazioni. Viene utilizzato per cercare, indicizzare, archiviare e analizzare dati di tutte le forme in tempo reale date le ottimizzazioni in termini di performance.

Kibana è la Graphical User Interface (GUI) progettata per *visualizzare e osservare, cercare, analizzare e gestire* i dati archiviati in Elasticsearch. Fornisce strumenti per creare dashboard, grafici, tabelle e mappe interattive, eseguire ricerche avanzate e monitorare in tempo reale i dati.

Elastic Agent rappresentano una singola e unificata modalità per aggiungere monitoring per log, metriche e altri tipi di dati a un host, semplificando e velocizzando l'implementazione nell'intera infrastruttura. Forniscono anche funzionalità di protezione dell'host da minacce di sicurezza. Elastic fornisce delle *Integration* per gli agent, ovvero una raccolta di asset che definiscono come collegarsi ad uno specifico prodotto o servizio esterno attraverso lo Stack Elastic, raccogliendo, archiviando e visualizzando facilmente qualsiasi dato da qualsiasi fonte. Forniscono inoltre risorse pronte all'uso come dashboard, visualizzazioni e pipeline per estrarre campi strutturati da log ed eventi provenienti da questi servizi. Infine, ogni agente ha una singola policy che può essere aggiornata per aggiungere integrazioni per nuove fonti di dati e protezioni di sicurezza.

Fleet è un componente che fornisce gestione centralizzata degli Elastic Agent e delle relative policy attraverso una specifica interfaccia utente di Kibana. Fleet funge da canale di comunicazione per gli Elastic Agent, i quali effettuano regolarmente il *check-in* per aggiornamenti e comunicando il relativo stato di salute. Quando viene apportata una

⁹<https://artifacthub.io/packages/helm/elastic/eck-stack>

modifica a una policy o ad una integrazione, tutti gli agenti ricevono l'aggiornamento durante il *check-in* successivo senza necessità di intervento manuale.

3.3.1 Deployment

Lo stack è reso disponibile tramite Helm¹⁰, un package-manager per Kubernetes che semplifica la definizione, l'installazione, l'upgrade e la gestione di applicazioni su un cluster. Si basa su *chart*, pacchetti preconfigurati di risorse Kubernetes che permettono di automatizzare e standardizzare il deployment di applicazioni, riducendo la complessità e migliorando la consistenza tra ambienti. Gli *helm chart* da deployare sono tre: **eck-operator**¹¹, **eck-stack**¹² e **kube-state-metrics**¹³.

eck-operator è il componente che si occupa di automatizzare il deployment, la gestione e l'orchestrazione di tutto lo stack ECK e quindi, nel nostro contesto, di Elasticsearch, Kibana e degli Elastic Agent. Il deployment non prevede configurazioni particolari e può essere fatto attraverso: `helm install my-eck-operator elastic/eck-operator`

eck-stack è il chart che contiene tutti i componenti deployabili dello stack; si pone come l'aggregazione di 6 chart corrispondenti ai 6 applicativi offerti. Nel dettaglio sono eck-elasticsearch, eck-kibana, eck-agent, eck-fleet-server, eck-logstash ed infine eck-apm-server.

La configurazione di default abilita solo Elasticsearch e Kibana, in quanto servizi essenziali dello stack, ma per il nostro caso di studio dovranno essere abilitati anche gli Agent e il Fleet Server. Dopo aver selezionato i servizi da deployare è necessario andare a configurare singolarmente ciascuno di essi, andando ad impostare opportunamente i *values* offerti dal relativo chart.

kube-state-metrics è un semplice servizio che ascolta il server API di Kubernetes e, senza modifiche, genera metriche sullo stato degli oggetti, concentrandosi sulla salute dei vari oggetti interni al cluster come nodi, deployment e pod; riflette lo *stato corrente del cluster* ed è utilizzato per monitorarlo. A differenza di *kubectl*, che applica certe euristiche ai dati, restituisce i dati grezzi, così come vengono ritornati dalle API di kubernetes. Si tratta di un sistema di monitoring di tipo Pull in quanto va ad esporre le metriche che collezione, come plain-text, su un endpoint HTTP, al *path*: `/metrics` sulla porta `8080`. Gli endpoint verranno interrogati dagli Agent per collezionarne i dati.

¹⁰<https://helm.sh/>

¹¹<https://artifacthub.io/packages/helm/elastic/eck-operator>

¹²<https://artifacthub.io/packages/helm/elastic/eck-stack>

¹³<https://artifacthub.io/packages/helm/bitnami/kube-state-metrics>

Il deployment non prevede configurazioni particolari e può essere fatto attraverso: `helm install my-kube-state-metrics bitnami/kube-state-metrics --version 4.2.12`

3.3.2 ECK-stack deployment values

In questa sezione verranno forniti e discussi i *values* utilizzati per la configurazione del Helm `eck-stack`.

Elasticsearch In questa configurazione illustrata nel Codice 3.1 viene specificata l'abilitazione e il nome da assegnare ai componenti che verranno deployati, successivamente viene specificata la struttura dei nodi.

Codice 3.1: Elasticsearch yaml values

```
eck-elasticsearch:
  enabled: true
  fullnameOverride: elasticsearch
  nodeSets:
    - config:
        node.store.allow_mmap: false
      count: 3
      name: default
```

Kibana Le specifiche di Kibana possiamo vederle come suddivise in due parti. La prima, visibile nel Codice 3.2, riguarda il servizio di Kibana vero e proprio specificandone l'abilitazione, il nome da assegnare al componente deployato e delle specifiche riguardanti il numero di repliche da creare e il riferimento ad Elasticsearch. Vengono infine specificate delle configurazioni riguardanti gli endpoint su cui si trovano il servizio di Elasticsearch e il Fleet Server.

Codice 3.2: Kibana yaml values

```
eck-kibana:
  enabled: true
  fullnameOverride: kibana
  spec:
    count: 1
    elasticsearchRef:
      name: elasticsearch
    config:
      xpack.fleet.agents.elasticsearch.hosts:
        ["https://elasticsearch-es-http.elastic-stack.svc:9200"]
      xpack.fleet.agents.fleet_server.hosts:
```

```
["https://fleet-server-agent-http.elastic-stack.svc:8220"]
```

La seconda, visibile nel Codice 3.3, invece, si focalizza sul fornire delle configurazioni per le policy che il Fleet Server (`id: eck-fleet-server`) e gli Agent (`id: eck-agent`) dovranno implementare. Le policy qui definite verranno utilizzate come values nella configurazione dei campi `policyID` dei chart `eck-agent` ed `eck-fleet-server`.

Codice 3.3: Kibana Policy yaml values

```
spec:
  config:
    xpack.fleet.agentPolicies:
      - name: Fleet Server on ECK policy
        id: eck-fleet-server
        namespace: default
        monitoring_enabled:
          - logs
          - metrics
        package_policies:
          - name: fleet_server-1
            id: fleet_server-1
            package:
              name: fleet_server
      - name: Elastic Agent on ECK policy
        id: eck-agent
        namespace: default
        monitoring_enabled:
          - logs
          - metrics
        unenroll_timeout: 900
        package_policies:
          - package:
              name: system
              name: system-1
          - package:
              name: kubernetes
              name: kubernetes-1
```

Elastic Agent In questa configurazione, illustrata nel Codice 3.4, viene specificata l'abilitazione e delle specifiche dei singoli agent come la policy che devono seguire (definita nelle configurazioni di kibana), il riferimento a Kibana, che fornisce automaticamente quello di Elasticsearch senza specificarlo, e infine il riferimento al Fleet Server con la

relativa modalità di esecuzione degli agenti *fleet-managed* (contrariamente alla modalità *stand-alone*)

Codice 3.4: Elastic Agent yaml values

```
eck-agent:
  enabled: true
  spec:
    policyID: eck-agent
    kibanaRef:
      name: kibana
    elasticsearchRefs: []
    fleetServerRef:
      name: fleet-server
    mode: fleet
```

Fleet server Configurazione molto semplice, visibile nel Codice 3.5, in cui viene specificata l'abilitazione, il nome da assegnare al componente deployato e delle specifiche riguardanti la policy da assegnare al server (definita nelle specifiche di Kibana) e i riferimenti a Kibana ed Elasticsearch

Codice 3.5: Fleet Server yaml values

```
eck-fleet-server:
  enabled: true
  fullnameOverride: "fleet-server"
  spec:
    policyID: eck-fleet-server
    kibanaRef:
      name: kibana
    elasticsearchRefs:
      - name: elasticsearch
```

3.3.3 Funzionamento

La Figura 3.1 mostra il flusso di esecuzione di ECK numerando le attività dalla 1 alla 8. Il flusso verrà percorso durante la seguente spiegazione.

All'avvio di *Kibana* le policy specificate nella configurazione, avvenuta tramite *helm chart*, vengono salvate in un indice, denominato *Fleet*, all'interno di Elasticsearch.

Il *Fleet Server*, all'avvio, stabilisce una connessione tramite i riferimenti posti tra i values con Kibana, per abilitare l'interfaccia grafica di gestione Fleet, e con Elasticsearch per ottenere i dati contenuti nell'indice di Fleet, ovvero le policy definite. In concreto, il

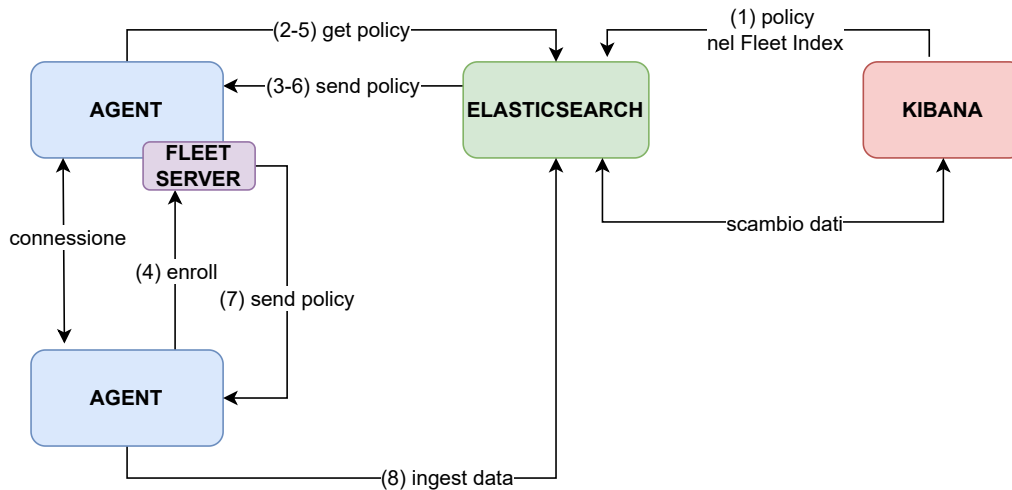


Figura 3.1: Diagramma del flusso di funzionamento dello stack

Fleet Server funziona come un agente normale, con la differenza che viene “enrollato” in una policy server.

Gli *Agenti*, all’avvio, essendo *fleet-enrolled* cercheranno di stabilire una connessione con il Fleet Server tramite il riferimento specificato. Andranno a richiedere al server la policy specificata nella loro configurazione (`policyID: eck-agent`) e Fleet si occuperà di recuperarla dall’indice di Elasticsearch per consegnarla al richiedente legittimo. Si dice quindi terminata la fase di *enrollment*.

Aprendo l’interfaccia di Fleet tramite Kibana è quindi possibile gestire le due policy caricate e i relativi agent che sono stati “enrollati”; le modifiche effettuate tramite il Fleet server alle policy vengono riflesse nell’indice di Elasticsearch e negli agent che seguono quella policy, effettuando un upgrade automatico. Viene mantenuta una connessione attiva tra agenti e fleet server per controllare lo stato di salute e la disponibilità di aggiornamenti; quando un agente si collega e c’è stato un cambiamento, il Fleet Server recupera da Elasticsearch la policy e la consegna all’agente effettuando così un aggiornamento autonomo.

Una volta che l’intera infrastruttura è stata deployata si hanno 3 agenti *fleet-enrolled*, uno per ciascuno nodo, che vanno ad usare le informazioni contenute nella policy per raccogliere i dati e inviarli ad Elasticsearch; gli agenti hanno un’architettura di tipo Push in quanto inviano attivamente i dati ad un endpoint che si occuperà di salvarli in appositi indici.

Tramite l’interfaccia di gestione delle policy è possibile scegliere quali Integrazioni aggiungere, andando così a raccogliere dati da una nuova sorgente e causando un rilascio di una nuova versione della rispettiva policy con conseguente aggiornamento degli agent. Oltre a svolgere operazioni di ricerca e analisi tramite la GUI di Kibana è possibile inviare query agli endpoint di Elasticsearch, i quali mettono a disposizione una serie di

API REST¹⁴.

3.4 Assurance Engine

L'applicativo viene sviluppato in Rust¹⁵, un linguaggio sicuro e ad alte prestazioni che previene molti errori comuni, fondandosi su 3 pacchetti principali quali Tokio¹⁶, per la gestione dell'esecuzione asincrona, Poem¹⁷, come framework web, e PoemOpenAPI¹⁸, per l'implementazione automatica di API e della relativa documentazione.

L'engine sviluppato si basa inoltre sui due concetti principali dell'Assurance Methodology, le **metriche** e i **contratti**, visti nella Sezione 2.4.1 e ora realizzati concretamente sotto forma di *Trait*; un componente Rust che definisce in modo astratto il comportamento/la funzionalità che un particolare tipo ha e può condividere con altri tipi.

La Figura 3.2 mostra uno schema che, partendo dai dati grezzi, illustra i passaggi chiave da compiere prima di arrivare ad un report del sistema.

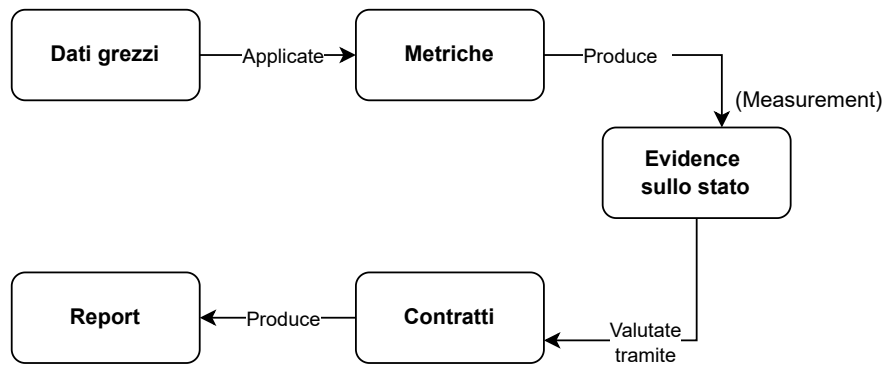


Figura 3.2: Diagramma del funzionamento

Le funzionalità sviluppate vengono poi rese accessibili tramite l'esposizione di **API rest** conformate ai criteri di *OpenAPI Specification v3*¹⁹; vengono quindi seguite alcune delle linee standard precedentemente identificate nella Sezione 2.2.2.

Segue l'illustrazione dei vari punti chiave nominati.

¹⁴<https://www.elastic.co/guide/en/elasticsearch/reference/current/rest-apis.html>

¹⁵<https://www.rust-lang.org/>

¹⁶<https://tokio.rs/>

¹⁷<https://crates.io/crates/poem>

¹⁸<https://crates.io/crates/poem-openapi>

¹⁹<https://github.com/OAI/OpenAPI-Specification/blob/main/versions/3.1.0.md>

3.4.1 Metric e Measurement

È stato definito il trait parametrico `Metric` come riportato nel Codice 3.6 che permette di specificare il tipo dell'argomento (`Arguments`) e dell'`Output`, i quali devono a loro volta soddisfare determinati *trait bounds* (`Send + Sync + ...`). Fornisce quindi un'interfaccia generica che codifica la struttura di una metrica definendo la firma di due metodi, `description()` e `measure()`. Il primo è semplicemente una funzione che ritorna una descrizione letterale della metrica. Il secondo, invece, è il metodo che concretamente va ad ottenere le misurazioni dagli endpoint di monitoring.

Codice 3.6: Metric Trait

```
#[async_trait]
pub trait Metric<Arguments, Output>
    where
        Arguments: Send + Sync + Type + ParseFromJSON + ToJSON,
        Output: Send + Sync + Type + ParseFromJSON + ToJSON,
    {
        fn id(&self) -> Box<str> {
            Box::from(std::any::type_name::<Self>())
        }

        fn description(&self) -> Box<str>;

        async fn measure(&self, args: Arguments)
            -> Result<Measurement<Arguments, Output>, Error>;
    }
}
```

L'implementazione concreta, in questo contesto, di quest'ultimo metodo va a costruire ed eseguire le query contro le API esposte dal servizio di Elasticsearch (endpoint) per ottenere i dati (*evidence*). Prende come input un argomento, *args*, del tipo generico da definire, `Arguments`, ovvero una struttura specifica che contiene tutti gli argomenti necessari a configurare la richiesta, include quindi sia la query vera e propria che dei parametri di configurazione come l'indice di interesse e la dimensione, come rappresentato nel Codice 3.7.

Codice 3.7: Arguments Struct

```
#[derive(Debug, Clone, PartialEq, serde::Deserialize,
        serde::Serialize, Object)]
pub struct BasicSearchArgs {
    pub query: Value,
    pub index: Vec<String>,
    pub from: Option<i64>,
    pub size: Option<i64>,
}
```

```
}
```

Measurement

In output, a meno di errori, il metodo `measure()` restituisce un `Measurement`, una struttura che rappresenta il risultato dalla misurazione, costituita dai campi `time`, `args` e `value` come mostrato nel Codice 3.8. Rispettivamente contengono l'istante di tempo in cui è avvenuta la misurazione, gli argomenti passati come input (`args` di tipo `Arguments`) e i valori ottenuti.

Codice 3.8: Measurement Struct

```
#[derive(Object)]
pub struct Measurement<Arguments, Output>
    where
        Arguments: Send + Sync + Type + ParseFromJSON + ToJSON,
        Output: Send + Sync + Type + ParseFromJSON + ToJSON,
{
    /// Time instant of measurement
    pub time: OffsetDateTime,
    /// Arguments in input
    pub args: Arguments,
    /// Value registered
    pub value: Output,
}
```

Qui entra in gioco il secondo parametro di tipo generico da definire, ovvero `Output`, che definisce la struttura del risultato da andare ad includere all'interno del `Measurement`, come si può vedere nel Codice 3.9 si tratta di un solo valore, `output`, che permette il contenimento di una stringa in formato json.

Codice 3.9: Output Struct

```
#[derive(Debug, Clone, PartialEq, serde::Deserialize,
    serde::Serialize, Object)]
pub struct BasicSearchOutput {
    pub output: Value,
}
```

3.4.2 Contract ed Evaluation

È stato definito il trait parametrico `Contract` come riportato nel Codice 3.10 che permette di specificare il tipo dell'argomento (`Arguments`), il quale deve a sua volta soddisfare determinati *trait bounds*. Fornisce quindi un'interfaccia generica che codifica la struttura di un contratto definendo la firma di due metodi, `description()` e `evaluate()`. Il primo, come per le metriche, è semplicemente una funzione che ritorna una descrizione letterale. Il secondo, invece, è il metodo che concretamente valuta le misurazioni raccolte combinando strutture di condizioni booleane.

Codice 3.10: Contract Trait

```
#[async_trait]
pub trait Contract<Arguments>
    where
        Arguments: Send + Sync + Type + ParseFromJSON + ToJSON,
{
    fn id(&self) -> Box<str> {
        Box::from(std::any::type_name::<Self>())
    }

    fn description(&self) -> Box<str>;

    async fn evaluate(&self, args: Arguments)
        -> Result<Evaluation<Arguments>, Error>;
}
```

L'implementazione concreta, in questo contesto, di quest'ultimo metodo va ad eseguire il metodo `measure()`, definito con l'implementazione del *trait metric* visto sopra, e a svolgere dei controlli sulle *misurazioni* che ottiene andando a verificare il superamento o meno di specifiche valutazioni, restituendo un valore booleano che ne definisce l'esito finale. Prende come input un argomento, `args`, del tipo generico da definire, `Arguments`, ovvero una struttura specifica i cui campi contengono dei valori di riferimento da confrontare con i valori delle misurazioni, per valutare il soddisfacimento o meno del contratto. Si potrebbero impostare, per esempio, dei valori che si desiderano di temperatura minimi, medi e massimi per poi controllare se le misurazioni ottenute rispettano i criteri specificati. Nel Codice 3.11 viene rappresentata la "struttura tipo".

Codice 3.11: Argument Contract Struct

```
#[derive(Debug, Clone, PartialEq, Serialize,
Deserialize, Object)]
pub struct SearchContractArgs {
    //values to check the fields with
}
```

Evaluation

In output, a meno di errori, il metodo `evaluate` restituisce un `Evaluation`, una struttura che rappresenta il risultato della valutazione, costituita dai campi `time`, `args` e `value` come mostrato nel Codice 3.12. Rispettivamente contengono l'istante di tempo in cui è avvenuta la misurazione, gli argomenti passati come input (*args* di tipo `Arguments`) e il valore booleano ottenuto.

Codice 3.12: Evaluation Struct

```
[derive(Debug, Clone, PartialEq, Serialize, Deserialize)]
pub struct Evaluation<Arguments>
    where
        Arguments: Send + Sync + Type + ParseFromJSON + ToJSON,
{
    /// Time entry in input
    pub time: OffsetDateTime,
    /// Arguments in input
    pub args: Arguments,
    /// Value verified as valid
    pub value: bool,
}
```

3.4.3 API

Come accennato prima le API vengono implementate attraverso il crate `PoemOpenAPI`, il quale garantisce che se il codice compila correttamente allora è pienamente conforme alle Specifiche di OpenAPI v3. Ogni funzionalità sviluppata, ovvero i metodi `measure()` per le Metriche ed `evaluate()` per i Contratti, viene mappata in uno specifico endpoint con il relativo Uniform Resource Locator (URL). Quando viene inviata una richiesta HTTP al *path* configurato, il server esegue la funzionalità corrispondente e restituisce l'esito. Per ogni funzionalità sviluppata, vengono esposti due endpoint. Il primo serve a gestire le richieste *POST* consentendo quindi l'esecuzione della funzionalità e l'ottenimento dei risultati; in concreto si tratta dell'effettiva esecuzione del metodo `measure()` (3.6) o `evaluate()` (3.10) definito. Il secondo, invece, serve per gestire le richieste *OPTIONS* fornendo una descrizione della funzionalità offerta dall'endpoint; in concreto esegue il metodo `description()` definito nella rispettiva implementazione. `Poem-OpenAPI` permette quindi di associare un *path* e un *method* (POST/OPTIONS) ad una funzione codificata.

I risultati, contenuti nelle strutture `Measurement` 3.8 ed `Evaluation` 3.12, vengono convertiti nelle enumerazioni `MeasureResponse` ed `EvaluationResponse` così da ritornare i risultati codificati in formato JSON se tutto è andato a buon fine (*status 200*) oppure, sempre in formato JSON, ritornare il tipo di errore (*400 input error; 500 system error*).

Per quanto concerne la **documentazione** delle API, necessaria per consentire agli utenti di interfacciarsi con esse, viene *generata automaticamente* dalla libreria in conformità allo standard di OpenAPI. Partendo dalla definizione delle funzioni delle API, PoemOpenAPI va a generare un file JSON contenente tutti i dettagli (la documentazione) e lo rende disponibile tramite uno specifico endpoint, modalità standard di esposizione della documentazione, evitando così la necessità di realizzarla manualmente.

Swagger UI

Viene integrata anche una pagina di gestione delle API generata automaticamente dalle specifiche OpenAPI, Swagger UI²⁰. Uno strumento open-source che permette di visualizzare e interagire con le API esposte direttamente dal browser attraverso un'interfaccia grafica, che consente di esplorarle e testarle facilmente senza necessità di scrivere codice. La Figura 3.3 mostra l'interfaccia grafica auto-generata da Swagger contenente tutte le API esposte, indicando metodo, *path* e una breve descrizione.

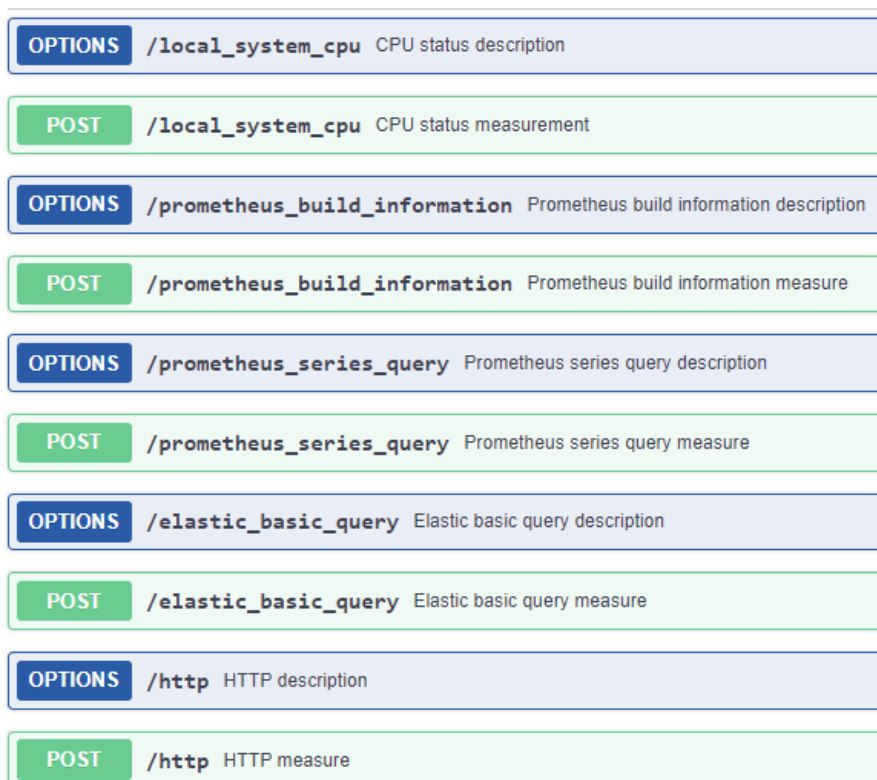


Figura 3.3: Interfaccia di Swagger contenente tutte le API esposte

²⁰<https://swagger.io/tools/swagger-ui/>

Andando ad interagire con l'interfacce proposta è possibile, tramite il menù a tendina, espandere la singola API vedendone i relativi dettagli. La Figura 3.4 mostra come deve essere costruito il body di una richiesta di quel tipo, andando a fornire un esempio di valori per quello specifico Schema. Tramite il bottone “*Try it out*” è possibile inviare una richiesta personalizzata su quello specifico endpoint.

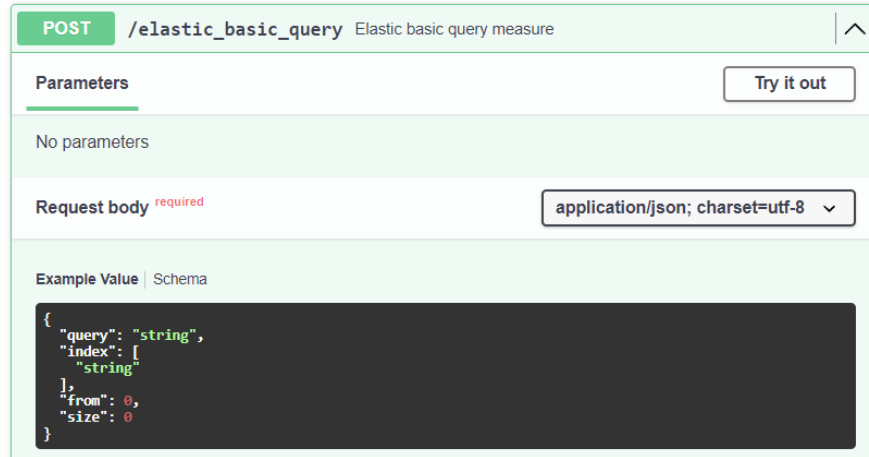


Figura 3.4: Interfaccia di Swagger contenente un esempio di schema per richieste Post

Si possono ottenere maggiori dettagli riguardo la struttura consultando la sezione Schema, come mostrato nella Figura 3.5, che fornisce anche il tipo associato ai campi.



Figura 3.5: Interfaccia di Swagger contenente lo schema json con i tipi dei valori

La Figura 3.6 mostra invece le possibili risposte, con i relativi *status code*, che si possono ottenere inviando una richiesta su uno specifico endpoint; viene anche qui specificato lo

schema della risposta in formato JSON.

Responses		
Code	Description	Links
200	Media type application/json; charset=utf-8 Controls Accept header. Example Value Schema <pre>{ "time": "2024-09-02T17:03:40.614Z", "args": { "query": "string", "index": { "string": }, "from": 0, "size": 0 }, "value": { "output": "string" } }</pre>	No links
400	Media type application/json; charset=utf-8 Example Value Schema <pre>{ "error": "string" }</pre>	No links
404		No links
500	Media type application/json; charset=utf-8 Example Value Schema <pre>{ "error": "string" }</pre>	No links

Figura 3.6: Interfaccia di Swagger contenente lo schema delle risposte alle richieste POST

3.4.4 Deployment e Vantaggi

L'*Assurance Engine* è stato progettato per essere eseguito all'interno di un cluster kubernetes, sfruttando la containerizzazione per garantire una distribuzione semplice e flessibile. In particolare, l'applicazione viene impacchettata in un container Docker (docker image) minimale contenente il binario eseguibile dell'engine che successivamente viene deployato all'interno del cluster Kubernetes. Una volta in esecuzione all'interno del cluster, l'applicativo cercherà di collegarsi agli altri servizi integrati nell'engine e disponibili all'interno dell'ecosistema, come Elasticsearch o Prometheus ad esempio.

Il sistema, per come è strutturato, si pone da *intermediario* tra l'utente finale e alcuni servizi di monitoring facendo in modo di ricevere le richieste dall'utente e successivamente indirizzarle all'apposito servizio. Questa infrastruttura può essere vista in un certo senso come un *request-reflector* che però, adottando questo design di sviluppo, fornisce svariati vantaggi tra cui la garanzia di alcune proprietà chiave per l'Assurance Engine, come la *scalabilità*, la *replicabilità* e l'*assenza di coordinazione aggiuntiva* in ambienti distribuiti.

Scalabilità e Replicabilità Queste importanti proprietà, derivanti dalla struttura dell'applicazione, consentono di deployare l'engine come una singola istanza o come istanze multiple, permettendo quindi di scegliere il numero di *repliche deployabili* per rendere il sistema distribuito.

L'architettura, pur essendo distribuita, non necessita alcuna coordinazione aggiuntiva tra le varie istanze/repliche in quanto tutte *equivalenti e intercambiabili* tra loro: le richieste possono essere inviate indistintamente a una qualsiasi istanza disponibile e verranno risolte in modo identico. Un *network load balancer*, come un *kubernetes service*, si occuperà di distribuire il traffico, in base alle condizioni della rete, tra le varie repliche disponibili. Questa semplicità è possibile perché il compito che ogni replica assolve è quello di intermediario *stateless* verso servizi di backend come Elasticsearch.

La Figura 3.7 mostra uno schema di un semplice cluster in cui sono state deployate 3 repliche dell'Assurance Engine poste dietro ad un network load balancer. Indipendentemente dalla replica a cui la richiesta verrà inoltrata dal load balancer, esse arriveranno sempre allo stesso servizio di backend.

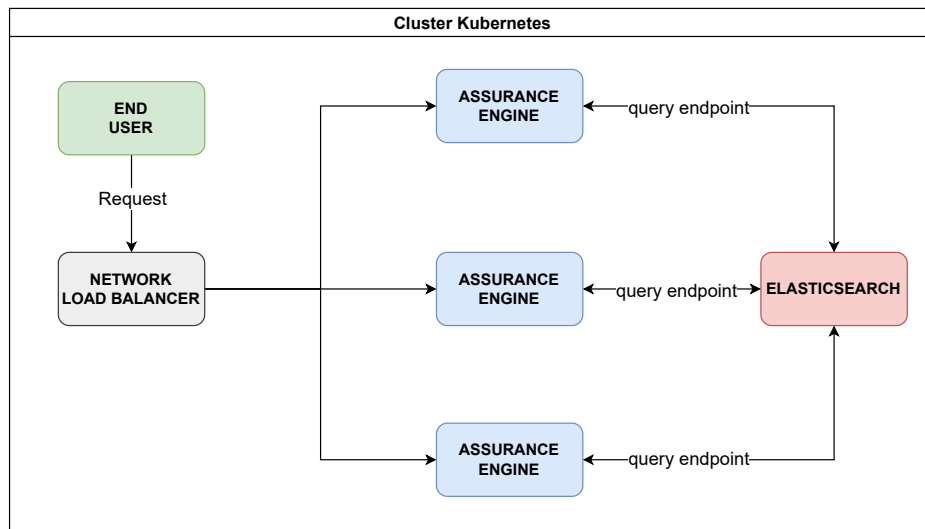


Figura 3.7: Diagramma Assurance Engine deployato su 3 nodi

Necessità di questo "framework intermediario"

Lo scopo di un sistema di assurance è garantire il soddisfacimento di determinate proprietà da parte di un sistema target.

La domanda che viene spontaneo porsi è quindi: *Come mai, essendo che Elasticsearch fornisce analisi per informazioni provenienti da una moltitudine di entità diverse, si è ricorsi ad un layer aggiuntivo (assurance engine), posto tra l'utente e il servizio di backend, per svolgere controlli di assurance?*

La risposta a questa domanda si trova nella definizione di sistema di assurance posta qua sopra; un sistema di assurance è quanto più completo e generico quante più proprietà e sistemi target eterogenei riesce a verificare.

Aggiungendo questo strato di interfacciamento è stato possibile accaparrarsi tutti i **vantaggi di Elasticsearch**, come la possibilità di collezionare dati da una moltitudine di servizi eterogenei grazie alle specifiche *Integration* e l'*efficienza* nel salvataggio e nella ricerca dei dati in un sistema distribuito. A questi, vanno ad aggiungersi i *vantaggi architetturali dell'assurance engine* illustrati nella Sezione precedente 3.4.4, che garantiscono **scalabilità** e **replicabilità**. Infine, il punto chiave che giustifica questo strato, si riconduce alle potenzialità che vengono offerte dall'utilizzo di un **linguaggio di programmazione** efficiente e affidabile come Rust. Utilizzando un vero e proprio linguaggio di programmazione invece che il limitato linguaggio di scripting *built-in* di Elasticsearch, è possibile avere una potenza espressiva maggiore sia per fare controlli più complessi (contratti) che per ottenere dati in modalità non implementate nei servizi di backend (*probe custom*). Alcuni esempi sono l'invio di semplici richieste HTTP verso un sito web per verificarne l'availability oppure l'invio di payload specifici su determinati endpoint per monitorarne la reazione; questo tipo di attività, possibile grazie all'impiego di un linguaggio di programmazione, permette quindi di analizzare contesti più generici. I servizi come Elasticsearch, invece, non consentono l'invio di richieste avanzate o *taylor-made* verso target specifici; il servizio recupera delle metriche dal target ma non ha una parte di "*probe custom*" che gli permette di ottenere informazioni specifiche.

In conclusione il vantaggio si può riassumere in una parola **genericità**, si possono combinare informazioni da varie entità diverse.

3.4.5 Funzionamento

Viene illustrato un esempio di ciclo completo di funzionamento. La Figura 3.8 mostra un'automa a stati finiti contenente lo schema del funzionamento illustrato tramite componenti grafici.

La prima operazione inizia dall'utente che invia una richiesta ad un endpoint esposto dal *Assurance Engine*, supponiamo per completezza il metodo POST di un contratto. Il server, quando riceve la richiesta, va ad eseguire il metodo `evaluate()` definito per quella funzionalità. L'esecuzione può essere suddivisa concettualmente nelle 3 parti focali pre-

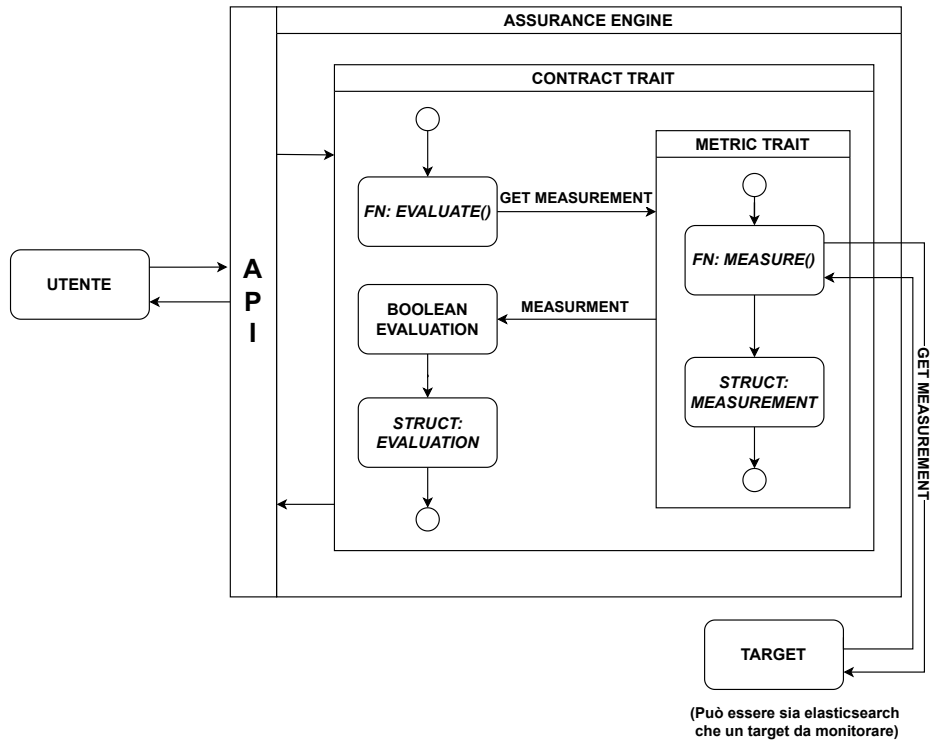


Figura 3.8: Assurance Engine: Flusso di esecuzione

cedentemente definite nell'*Assurance Process* definito nella Sezione 2.4.2; rispettivamente sono *Measurement collection*, *Contract evaluation* e *Report composition*.

Measurement collection In questa fase viene eseguito il metodo `measure()` precedentemente illustrato nella Sezione 3.4.1 che si occupa di ottenere delle metriche da un target. Come accennato prima, grazie alla genericità di cui gode, il metodo `measure()` può andare ad interrogare un servizio di backend come Elasticsearch o inviare richieste custom. I dati ottenuti vengono trattati come `measurements` e di fatto costituiscono le *evidence* o prove necessarie ai contratti per stilare una valutazione del sistema; seguono il modello definito nella Sezione 3.4.1. Viene effettuata quindi una fase di parsing della risposta JSON ottenuta all'interno del *measurement* andando ad estrarre i campi e i valori desiderati; vengono svolte anche operazioni di aggregazione dei dati come somme, medie, ecc (*metric evaluation*). Una volta ottenute le evidence definitive inizia la seconda fase.

Contract evaluation Questa fase è concettualmente molto semplice ma racchiude al suo interno la potenzialità e le garanzie che questo sistema può offrire. Si tratta di

controlli booleani effettuati tra le *measurements* ottenute facendo *metric evaluation* e gli argomenti passati come parametri al contratto, precedentemente visti nella Sezione 3.4.2 alla Figura 3.11. I risultati dei vari controlli singoli sulle metriche vengono successivamente aggregati in un'unico risultato finale che dichiara il soddisfacimento o meno della proprietà che si stava verificando.

Report composition L'ultima fase si tratta semplicemente di ritornare i risultati ottenuti in modo formale e valutabile a posteriori. Viene quindi ritornata la struttura definita nella Sezione 3.4.2 contenente sia l'esito che gli argomenti usati per fare la valutazione, in aggiunta al *timestamp*.

L'utente che ha inviato la richiesta riceve una risposta in formato JSON, contenente o la struttura sopra definita oppure una descrizione della tipologia dell'errore che è incorso.

3.4.6 Esempi di richieste: Elastic e SQL Query

Di seguito vengono proposti tre esempi di utilizzo del metodo `measure()`, per raccogliere metriche da Elasticsearch, con tre query differenti in formato JSON. Le prime due sfruttano il linguaggio di ricerca fornito da Elasticsearch, un completo Query Domain Specific Language (DSL) basato su JSON, mentre la seconda permette di effettuare ricerche usando query SQL e ritornando i risultati in forma tabellare.

L'ultimo esempio mostra uno schema generale di invocazione del metodo `measure()` con una generica query.

Esempio 3.4.1. Esempio di ricerca che colleziona **tutti i dati** presenti nell'indice e li ordina cronologicamente sulla base dei campi Date e Time del dataset. Di seguito, nel Codice 3.13, viene mostrata la query in formato JSON necessaria per svolgere tale operazione.

Codice 3.13: Valori JSON per query *match all*

```
1  "query": {
2    "match_all": {}
3  },
4  "sort": [
5    {"Date": {"order": "asc"}},
6    {"Time": {"order": "asc"}}
7  ],
```

Esempio 3.4.2. Esempio di ricerca che colleziona i dati presenti nell'indice il cui **intervallo temporale**, campi Date e Time, rispetta un certo criterio e li ordina cronologica-

mente. Di seguito, nel Codice 3.14, viene mostrata la query in formato JSON necessaria per svolgere tale operazione.

Codice 3.14: Valori JSON per query *range* su intervalli temporali

```

1  "query": {
2    "range": {
3      "Date": {
4        "gte": "11/03/2004",
5        "lte": "15/03/2004"
6      },
7      "Time": {
8        "gte": "18.00.00",
9        "lte": "24.00.00"
10     }
11   },
12 },
13 "sort": [
14   {"Date": {"order": "asc"}},
15   {"Time": {"order": "asc"}}
16 ],

```

Esempio 3.4.3. Esempio di ricerca eseguito tramite query **SQL** che estrae due campi dall'indice filtrando per il campo Date. Nel Codice 3.15, viene mostrata la query in formato JSON necessaria per svolgere tale operazione.

Codice 3.15: Valori JSON per query *SQL*

```

1  "query": "SELECT T, \"N02(GT)\"
2          FROM imported_dataset
3          WHERE Date = '05/10/2004'"

```

Esempio 3.4.4. Nel seguente Codice 3.16 viene mostrata l'invocazione del metodo `measure()` con la relativa generica query specificabile tramite valori JSON nel Query DSL di Elastic.

Codice 3.16: Esecuzione Rust della query nel linguaggio di Elastic

```

client.measure(BasicSearchArgs {
  query: json!({<json_values>}),
  index: vec!["imported_dataset".to_string()],
  from: None,
  // size: Some(1)
  size: None,
})

```

Mentre in questo Codice 3.17 viene mostrata l'invocazione del metodo `measure()` con la relativa generica query SQL specificabile tramite valori JSON.

Codice 3.17: Esecuzione Rust della query SQL

```
client.measure(ArgsSQL {  
  query: json!({<json_values>})  
})
```

Capitolo 4

Esperimenti

In questo capitolo verranno presentati i test condotti sulle performance del sistema di *Assurance Engine* collegato ad Elasticsearch. L'obiettivo è quello di descrivere i tipi di test/benchmark eseguiti e i risultati oggettivi ottenuti tramite grafici e tabelle riepilogative. Questi benchmark sono stati selezionati perché rappresentano i principali casi d'uso nel contesto di misurazione e valutazione delle performance. In altre parole, sono stati analizzati quegli aspetti statistici chiave, cruciali per comprendere e ottimizzare le prestazioni complessive del sistema. I benchmark sono stati condotti utilizzando Criterion¹, una libreria di benchmarking utilizzata per misurare e analizzare prestazioni di esecuzione di specifiche funzioni per il linguaggio di programmazione Rust.

4.1 Statistiche offerte

L'implementazione di benchmark tramite Criterion fornisce delle tabelle, denominate *additional statistics*, e dei diagrammi per poter analizzare e confrontare i risultati ottenuti. Vengono elencate di seguito tutte le *additional statistics* ottenibili con una breve illustrazione. È da considerare che tutte le valutazioni vengono espresse tramite intervalli di confidenza, seguono quindi la forma [`<lower bound>` `<estimated>` `<upper bound>`].

- **R2** Coefficiente di Determinazione. È una misura della bontà di adattamento di un modello di regressione lineare ai dati; misura quindi la correttezza del modello statistico utilizzato. Se il valore R2 è basso, questo potrebbe indicare che il benchmark non sta eseguendo la stessa quantità di lavoro a ogni iterazione.
- **Mean** Media dei tempi per iterazione

¹<https://bheisler.github.io/criterion.rs/book/>

- **Std. Dev.** Deviazione Standard dei tempi per iterazione. È una misura della dispersione o variabilità di un insieme di dati che indica quanto i valori di un dataset si discostano in media dalla loro media aritmetica.
Se è più grande rispetto agli altri valori di tempo significa che i benchmark sono rumorosi e potrebbe essere necessario modificarli per ridurre il rumore.
- **Median** Mediana. È una misura che indica il valore centrale di una distribuzione; divide un dataset ordinato in due metà uguali contenenti lo stesso numero di valori (o quasi).
- **Median Absolute Deviation (MAD)** Deviazione Assoluta Mediana. È una misura di dispersione che quantifica quanto i dati si discostano dalla loro mediana. È un'alternativa alla deviazione standard e viene utilizzata per misurare la variabilità dei dati in modo che sia meno influenzata dai valori estremi o outliers.
Se la deviazione assoluta mediana è ampia indica che i parametri di riferimento sono rumorosi.
- **Outliers** Valori anomali. Vengono rilevate misurazioni insolitamente sproporzionate rispetto alle restanti, sia valori troppo alti che troppo bassi.
Se sono presenti molti valori anomali, la misurazione ci suggerisce che i risultati del benchmark sono rumorosi e non pienamente affidabili.

Oltre ai dati presentati in forma tabellare (ed immagazzinati in formato JSON), Criterion mette a disposizione vari diagrammi o *plot* per rappresentare graficamente le performance.

- **Violin plot** Diagramma a violino. Una rappresentazione statistica che descrive distribuzioni di probabilità utilizzando curve di densità; viene utilizzato per confrontare distribuzioni e quindi confrontare i benchmark.
- **Density plot** Diagramma della densità. Mostra il tempo medio per iterazione. Nel diagramma si ha una regione colorata che rappresenta la probabilità che un iterazione richieda quella quantità di tempo e una linea marcata che identifica la media.
- **Time/Iteration plot** Mostra il tempo medio di iterazione impiegato dai diversi sample o tentativi. Ogni tentativo (generalmente da 0 a 100) viene rappresentato da un puntino e l'altezza ne definisce il tempo utilizzato.

4.2 Benchmark

I benchmark scritti utilizzando il framework Criterion vengono eseguiti tramite il comando `cargo bench -p ae-probes --bench elastic_probes`. L'esecuzione di ogni benchmark viene suddivisa in quattro fasi.

1. **Warm-up**: la funzione sottoposta al benchmark viene eseguita ripetutamente per un periodo di tempo configurabile, di default 3 secondi. Serve a riscaldare le cache del processore e (se applicabile) le cache del file system per adattarsi al nuovo carico di lavoro.
2. **Measurement**: la funzione sottoposta al benchmark viene eseguita ripetutamente per generare una stima del tempo impiegato da ogni iterazione; ogni iterazione viene definita come un sample e contiene le rispettive misurazioni.
3. **Analysis**: i sample vengono analizzati per calcolare le statistiche che vengono riportate, salvate e disegnate.
4. **Comparison**: il sample corrente viene confrontato con il sample ottenuto nell'iterazione del benchmark precedente.

Terminate queste fasi e quindi terminato il benchmark viene generata una struttura di directory all'interno del progetto Rust sotto `assurance-engine/target/criterion/<BENCHMARK_NAME>` come mostrato nella Figura 4.1.

La cartella `/new` contiene le statistiche dell'ultimo benchmark eseguito mentre la cartella `/base` include le statistiche del benchmark precedente oppure è rimpiazzata da cartelle con il nome dei benchmark salvati. Tramite il comando `cargo bench --bench elastic_probes -- --save-baseline <nome>` è possibile andare a salvare un benchmark con un nome e tramite il comando `cargo bench --bench elastic_probes --baseline <nome>` è possibile selezionarlo come benchmark per il confronto. È stato salvato un benchmark eseguito localmente nel laboratorio.

Il comando utilizzato per ottenere le statistiche dettagliate delle valutazioni in locale e confrontarle graficamente con le altre salvate precedentemente è il seguente: `cargo bench -p ae-elastic-probe --bench elastic_probes -- --load-baseline locale --baseline old`.

La cartella `/report` contiene tutti i diagrammi, sia quelli generati del singolo benchmark che quelli del confronto con uno precedente, contenuti nella sotto-cartella `/both`. I risultati tramite interfaccia grafica sono consultabili aprendo il file `/report/index.html`.

Il benchmark effettuato valuta concretamente l'esecuzione di 4 funzione suddivise a copie. Le prime due utilizzano la ricerca tramite Query DSL proposta nel Codice 3.13 che estrae tutti gli elementi e li ordina cronologicamente. Le altre due, invece, utilizzano

```

BENCHMARK_NAME/
├── base/
│   ├── benchmark.json
│   ├── estimates.json
│   ├── sample.json
│   └── tukey.json
├── change/
│   └── estimates.json
├── new/
│   ├── benchmark.json
│   ├── estimates.json
│   ├── sample.json
│   └── tukey.json
└── report/
    ├── both/
    │   ├── pdf.svg
    │   ├── iteration_times.svg
    │   └── regression.svg (optional -> not present)
    ├── change/
    │   ├── mean.svg
    │   ├── median.svg
    │   └── t-test.svg
    ├── index.html
    ├── MAD.svg
    ├── mean.svg
    ├── median.svg
    ├── SD.svg
    ├── slope.svg (optional -> not present)
    ├── pdf.svg
    ├── pdf_small.svg
    ├── relative_pdf_small.svg
    ├── iteration_times.svg (optional -> present)
    ├── iteration_times_small.svg (optional -> present)
    ├── relative_iteration_times_small.svg (optional -> present)
    ├── regression.svg (optional -> not present)
    ├── regression_small.svg (optional -> not present)
    └── relative_regression_small.svg (optional -> not present)

```

Figura 4.1: Directory tree generato dai benchmark di Criterion

la ricerca tramite query SQL proposta nel Codice 3.15 che estrae due campi dall'indice filtrando sulla data. Ogni coppia è costituita da una funzione `measure()` per le Metriche e da una funzione `evaluate()` per i Contratti.

Di seguito, per ogni tipo di ricerca (SEARCH e SQL), viene proposto prima un confronto tra l'esecuzione dei due metodi, `measure` ed `evaluate`, tramite un diagramma a violino e poi un'analisi dettagliata dei metodi presi singolarmente.

4.3 SEARCH Benchmark

Benchmark effettuato sulle funzioni `measure()` ed `evaluate()` basate sulla Query DSL proposta nel Codice 3.13 che estrae tutti gli elementi e li ordina cronologicamente.

Confronto funzioni benchate La Figura 4.2 mostra due distribuzioni di probabilità indicanti il tempo medio di esecuzione delle funzioni `evaluate` e `measure`. L'asse delle ascisse rappresenta il tempo di esecuzione, mentre quello delle ordinate i due benchmark da confrontare. Si osserva come le distribuzioni si concentrino intorno al valore medio, 6 ms per le metriche e 5 ms per i contratti. Tuttavia, si nota come l'ampiezza del benchmark di Metric si discosti notevolmente dal valore medio a causa di alcuni *outliers* misurati, sopra i 9 ms; sono identificati da una riga orizzontale che quindi ne definisce una probabilità trascurabile. In conclusione, l'esecuzione del benchmark sui contratti è stata omogenea in quanto tutte le misurazioni hanno ottenuto valori addensati intorno ai 5 ms. Nel caso dei benchmark sulle metriche, invece, si nota una media spostata leggermente verso destra, 6 ms, poiché influenzata da misurazioni outliers.

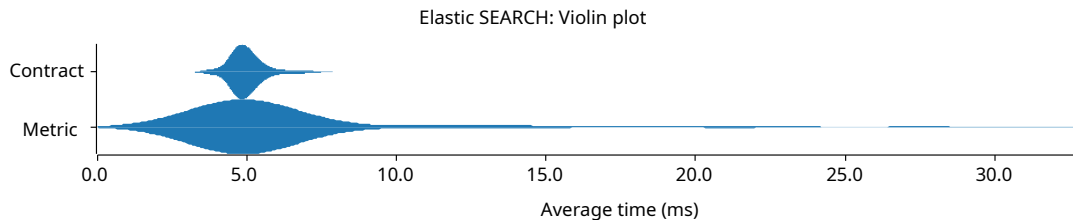


Figura 4.2: Diagramma a violino richieste di tipo SEARCH

4.3.1 Metric - `measure()`

Diagramma distribuzione probabilità-tempo La Figura 4.3 illustra la distribuzione di probabilità relativa al tempo necessario per eseguire il metodo `measure`. Si possono osservare quattro picchi nel diagramma. Il primo, nonché più significativo, determina prevalentemente il comportamento di questa distribuzione, in quanto possiede l'altezza massima e quindi una densità di probabilità maggiore; il picco si trova sul valore 5 ms.

Gli altri altri 3 picchi identificati sono di rilevanza inferiore più ci si allontana dal valore del picco massimo. Questi ultimi si trovano oltre al valore definito come limite massimo per decretare un valore conforme agli altri e quindi non outlier; nel diagramma viene rappresentato da una linea rossa a 9 ms, “severe outliers”. Gli outlier sono identificati da puntini rossi mentre la media è rappresentata da una linea blu a 6.2 ms.

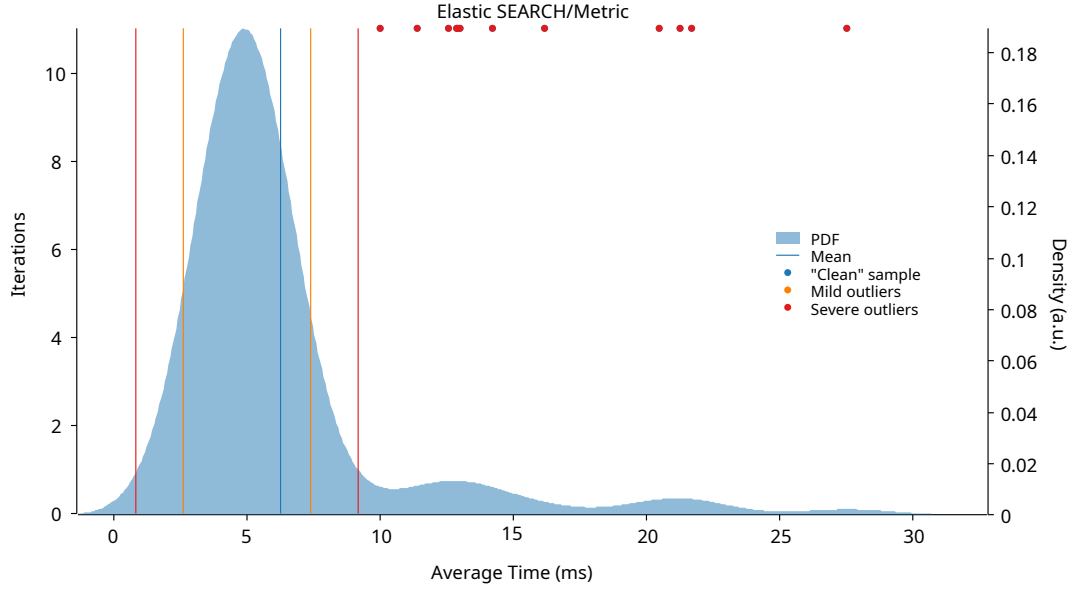


Figura 4.3: Diagramma densità probabilistica richieste `measure` di tipo SEARCH

Statistiche calcolate Nella Tabella 4.1 vengono mostrate le statistiche ottenute tramite i benchmark. Si nota una media del tempo di esecuzione pari a 6.2 ms e una deviazione standard rispetto tale valore pari a 4.1 ms.

Tabella 4.1: Valutazioni statistiche SEARCH/Metric benchmark

Misurazione	<i>Lower bound</i>	Estimate	<i>Upper bound</i>
Mean	5.5126 ms	6.2485 ms	7.1098 ms
Std. Dev.	2.5683 ms	4.1361ms	5.4378 ms
Median	4.8315 ms	5.0052 ms	5.1727 ms
MAD	691.05 μ s	906.15 μs	1.0739 ms
R2	0.0005357	0.0005529	0.0005297

Diagramma Sample-Time La Figura 4.4 rappresenta un diagramma che mostra il tempo che ogni sample ha impiegato per terminare l'esecuzione della funzione. Sono presenti 100 sample e l'altezza ne indica il tempo impiegato. Si nota come le prime 14 iterazioni hanno tempi maggiori o uguali ai 10 ms e quindi, essendo maggiori della soglia di normalità (9 ms), vengono identificati come outliers. Le iterazioni successive si stabilizzano intorno ai 5 ms.

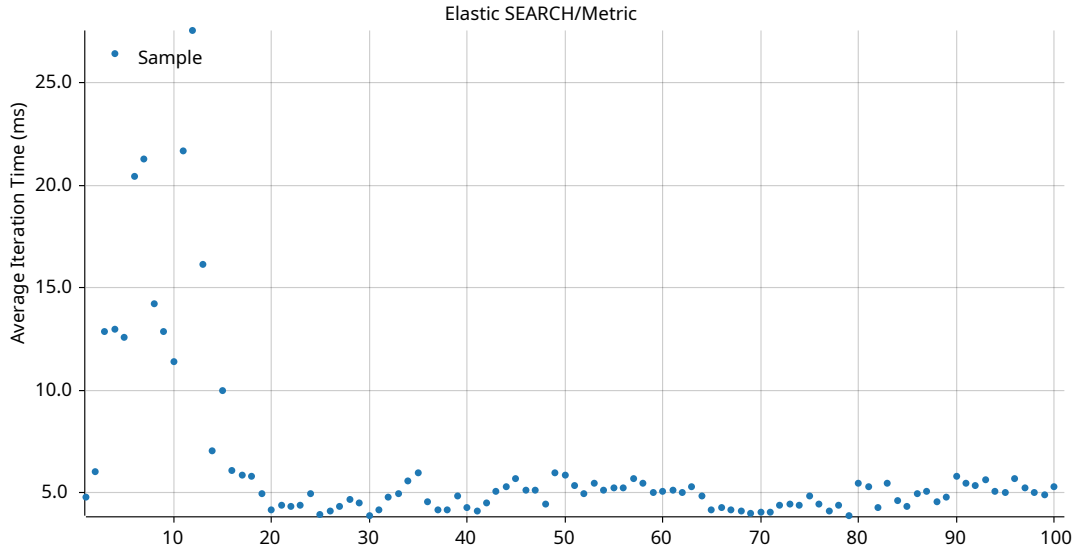


Figura 4.4: Diagramma iteration/time richieste `measure` di tipo `SEARCH`

4.3.2 Contract - `evaluate()`

Diagramma distribuzione probabilità-tempo La Figura 4.5 presenta la distribuzione di probabilità del tempo di esecuzione del metodo `evaluate`. Si nota la presenza di un singolo picco centrale che si discosta leggermente dalla media, il cui valore è 5 ms. Questo scostamento è causato dalla presenza di alcuni outliers lievi, che si trovano oltre i 6.1 ms e influenzano leggermente la valutazione finale. La concentrazione massima di densità risiede tra i valori temporali 4.5 e 5.5.

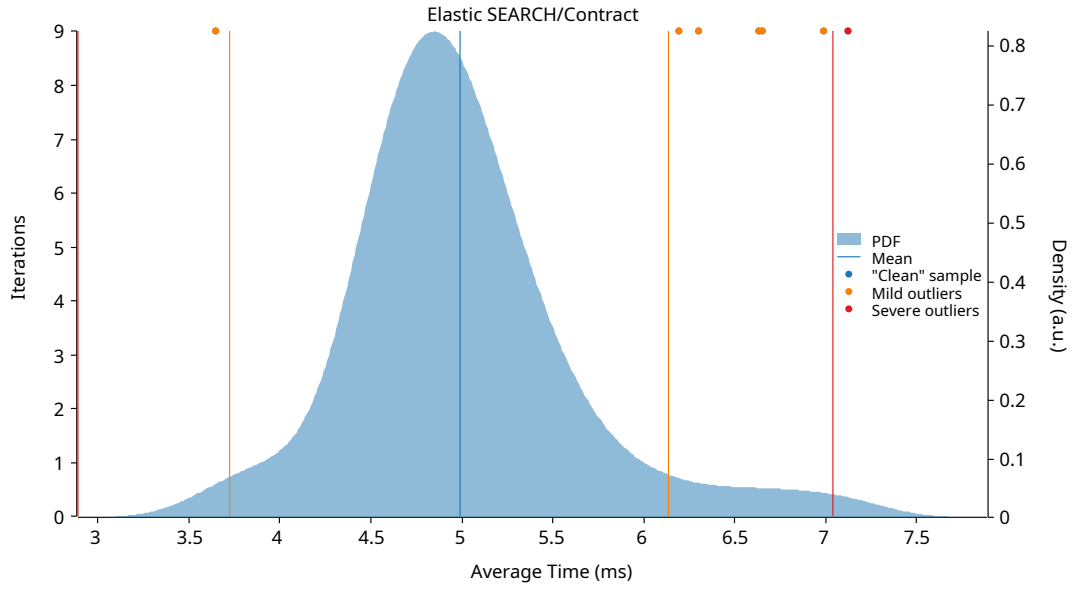


Figura 4.5: Diagramma densità probabilistica richieste **evaluate** di tipo SEARCH

Statistiche calcolate La Tabella 4.2 riporta le statistiche derivanti dai benchmark. Si evidenzia un tempo medio di esecuzione pari a 4.9 ms e una deviazione standard pari a 600.06 μ s, il che indica misurazioni molto stabili.

Tabella 4.2: Valutazioni statistiche SEARCH/Contract benchmark

Misurazione	<i>Lower bound</i>	Estimate	<i>Upper bound</i>
Mean	4.8786 ms	4.9931 ms	5.1138 ms
Std. Dev.	468.76 μ s	600.06 μs	716.74 μ s
Median	4.8183 ms	4.9212 ms	4.9911 ms
MAD	312.40 μ s	446.40 μs	544.85 μ s
R2	0.0054919	0.0056929	0.0054707

Diagramma Sample-Time La Figura 4.6 illustra un diagramma che descrive il tempo impiegato da ciascun sample per completare l'esecuzione della funzione. Sono riportati 100 sample e l'altezza ne indica il tempo impiegato. Non si evidenzia nessuna dispersione significativa nelle valutazioni dei sample, sono presenti 6 outlier al di sopra del 6. Si nota come la condensazione massima di valori sia nell'intervallo temporale 4.5 e 5.5.

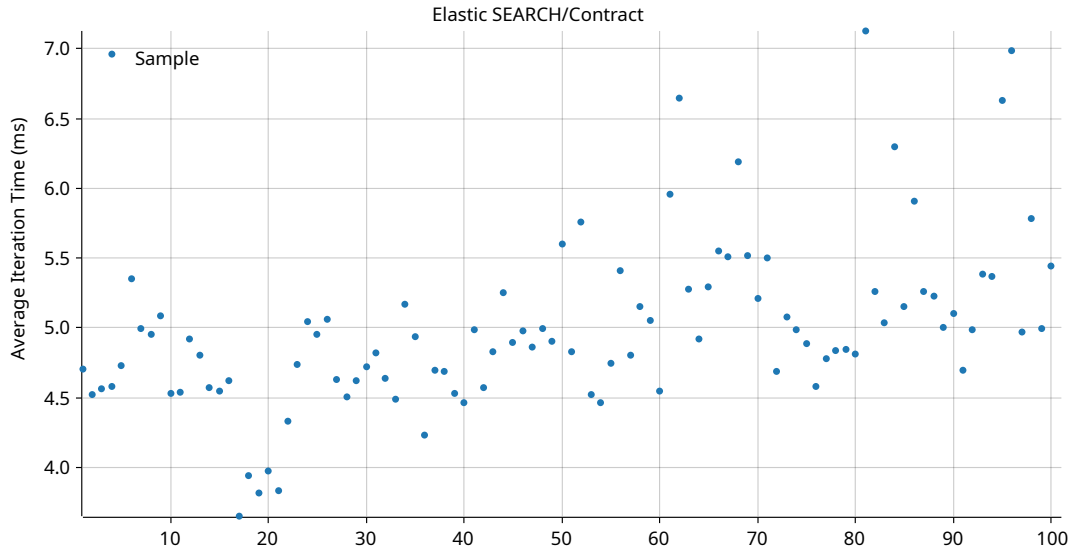


Figura 4.6: Diagramma iteration/time richieste `evaluate` di tipo SEARCH

4.4 SQL Benchmark

Benchmark condotto sulle funzioni `measure()` ed `evaluate()` utilizzando la Query SQL descritta nel Codice 3.15 che estrae due campi dall'indice filtrando sulla data.

Confronto funzioni benchate La Figura 4.7 illustra due distribuzioni di probabilità relative al tempo medio di esecuzione delle funzioni `evaluate` e `measure`. Come nel precedente diagramma a violino, sull'asse delle ascisse è presente il tempo di esecuzione mentre su quello delle ordinate i due benchmark da confrontare. Si può notare come le distribuzioni si concentrino intorno al valore medio, 8.5 ms per le metriche e 7.6 ms per i contratti. Si nota anche come l'ampiezza di entrambi i benchmark, ma prevalentemente di quello Metric, si discosti dal valore medio a causa di alcuni *outliers* misurati, identificati ancora da una riga orizzontale che ne definisce una probabilità irrisoria. Per concludere, l'esecuzione del benchmark sui contratti è ancora una volta stabile in quanto tutte le misurazioni hanno ottenuto valori addensati intorno ai 7.6 ms, ad eccezione di lievi outlier. Il benchmark sulle metriche, come osservato nei benchmark SEARCH, mostra una media leggermente più alta, intorno agli 8.5 ms, a causa di misurazioni outliers più frequenti.

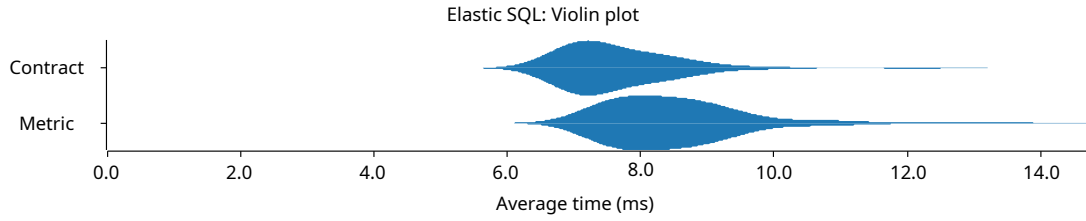


Figura 4.7: Diagramma a violino richieste di tipo SQL

4.4.1 Metric - `measure()`

Diagramma distribuzione probabilità-tempo La Figura 4.8 mostra la distribuzione di probabilità sulle tempistiche di esecuzione del metodo `measure`. Si evidenzia anche qui la presenza di un unico picco centrale, che si allontana leggermente dalla media di 8.5 ms a causa della presenza di lievi outliers, al di sopra dei 10.8 ms, che influiscono lievemente sulla valutazione complessiva. La maggior parte dei dati è concentrata tra 7 e 10 ms.

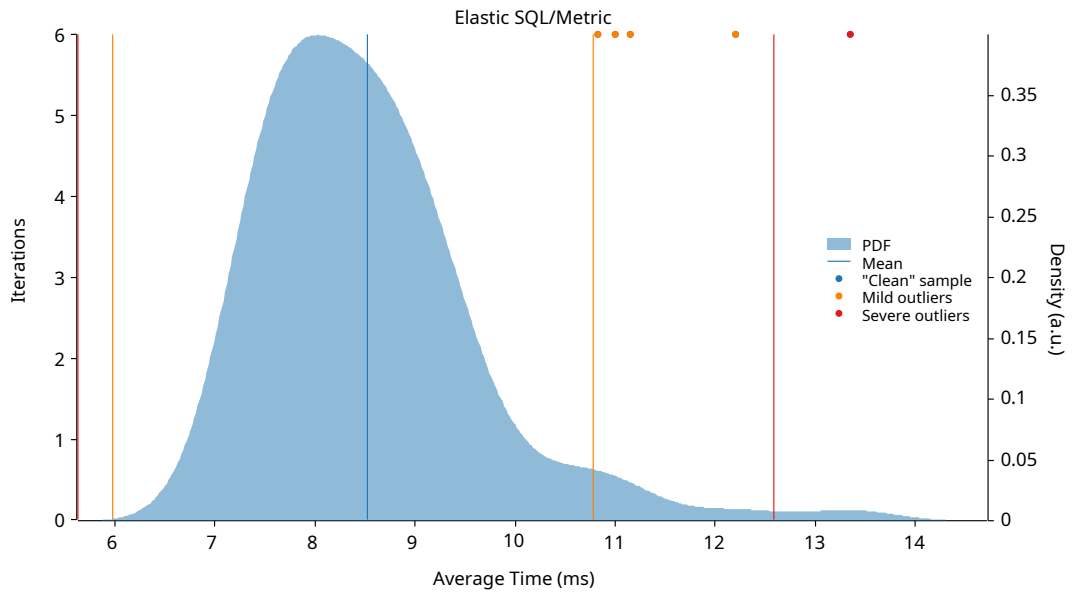


Figura 4.8: Diagramma densità probabilistica richieste `measure` di tipo SQL

Statistiche calcolate Nella Tabella 4.3 sono elencate le statistiche ottenute tramite i benchmark, con una media del tempo di esecuzione pari a 8.5 ms e una deviazione standard rispetto tale valore pari a 1.07 ms.

Tabella 4.3: Valutazioni statistiche SQL/Metric benchmark

Misurazione	<i>Lower bound</i>	Estimate	<i>Upper bound</i>
Mean	8.3206 ms	8.5206 ms	8.7393 ms
Std. Dev.	812.45 μ s	1.0723 ms	1.3265 ms
Median	8.0902 ms	8.3803 ms	8.6017 ms
MAD	737.19 μ s	895.16 μs	1.1394 ms
R2	0.0040924	0.0042356	0.0040656

Diagramma Sample-Time La Figura 4.9 presenta un diagramma che raffigura i tempi di esecuzione della funzione per ciascun sample. Tra i 100 sample presenti non emerge nessuna dispersione significativa nelle valutazioni; sono presenti 5 outlier al di sopra del 10.8. Si evidenzia come la maggior parte dei valori è concentrata nell'intervallo temporale 7 e 10.

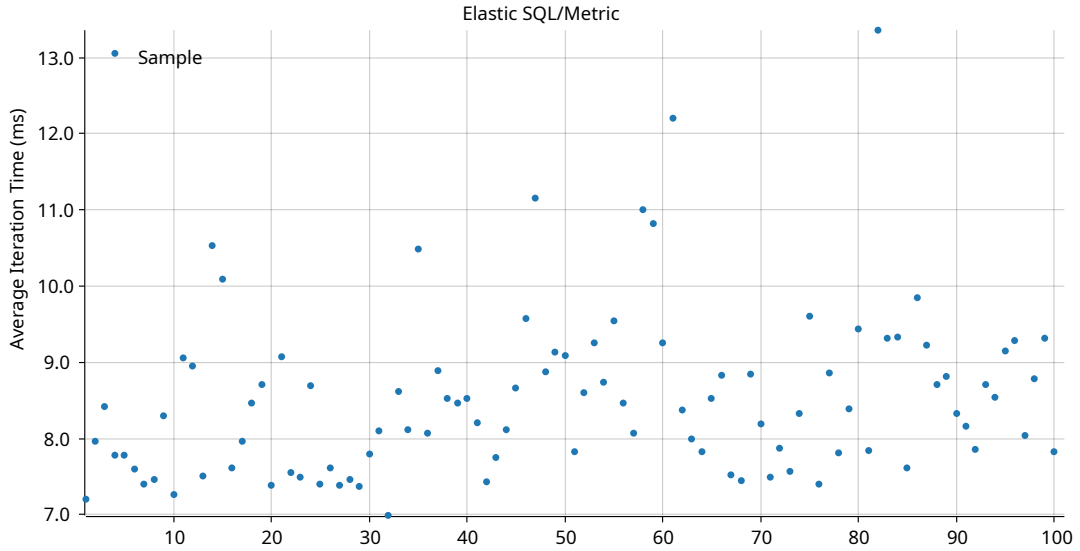


Figura 4.9: Diagramma iteration/time richieste `measure` di tipo SQL

4.4.2 Contract - `evaluate()`

Diagramma distribuzione probabilità-tempo La Figura 4.10 rappresenta la distribuzione di probabilità del tempo richiesto per l'esecuzione del metodo `evaluate`. Si osserva un unico picco centrale che si discosta leggermente dalla media, il cui valore è 7.6 ms. Questa variazione è ancora una volta influenzata dalla presenza di outliers sopra

i 9.8 ms, che seppur presenti in minima frequenza hanno un impatto sulla valutazione finale. La concentrazione principale dei valori si colloca tra 6.5 e 9 ms; si notano anche i due valori outlier al di sopra dei 10 ms.

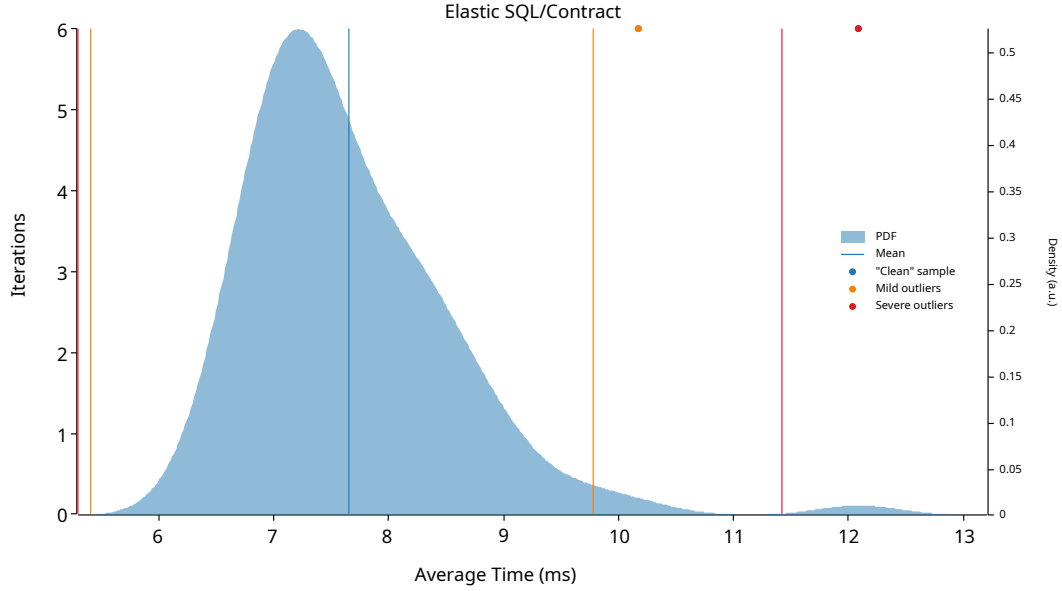


Figura 4.10: Diagramma densità probabilistica richieste `evaluate` di tipo SQL

Statistiche calcolate La Tabella 4.4 mostra i risultati statistici dei benchmark effettuati, i quali mostrano una media del tempo di esecuzione pari a 7.6 ms e una deviazione standard rispetto tale valore pari a 882.01 μ s.

Tabella 4.4: Valutazioni statistiche SQL/Contract benchmark

Misurazione	<i>Lower bound</i>	Estimate	<i>Upper bound</i>
Mean	7.4923 ms	7.6578 ms	7.8354 ms
Std. Dev.	666.71 μ s	882.01 μs	1.1105 ms
Median	7.3017 ms	7.4661 ms	7.6194 ms
MAD	505.58 μ s	729.81 μs	898.74 μ s
R2	0.0102391	0.0105995	0.0101868

Diagramma Sample-Time Nella Figura 4.11 viene raffigurato un diagramma che rappresenta il tempo di esecuzione di ciascun sample per terminare la funzione. Su 100 sample presenti, di cui l'altezza ne indica il tempo impiegato, non si nota nessuna dispersione significativa nelle valutazioni. É il benchmark con il minor numero di outlier, solo

2 sono presenti sopra al valore 9.8. I valori si condensano tutti nell'intervallo temporale 6.5 e 9.

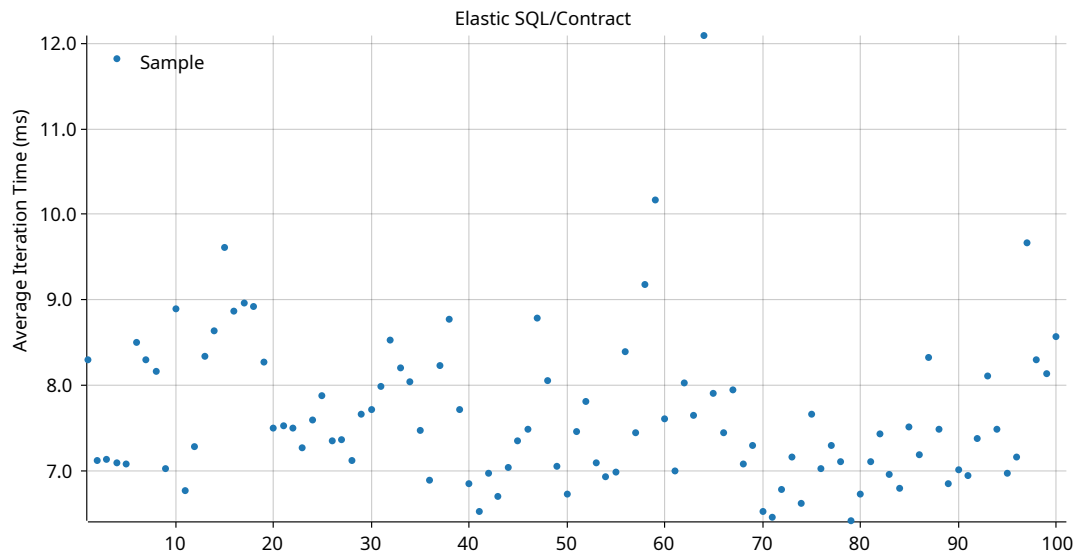


Figura 4.11: Diagramma iteration/time richieste **evaluate** di tipo SQL

Capitolo 5

Discussione

In questa sezione verranno analizzati i risultati ottenuti dai test sulle performance del sistema illustrati nel capitolo precedente. L'obiettivo è quello di fornire un commento sui risultati ottenuti e sulle motivazioni di situazioni anomale.

5.1 SEARCH Benchmark

Verrà effettuato un breve commento sui risultati e sui diagrammi ottenuti dai benchmark di SEARCH.

Metric - measure() Il benchmark visto nella Sezione 4.3.1 con la relativa Figura 4.3 è quello con il maggior numero di outlier, 14/100, tutti di tipo “severe outlier”. È interessante notare dalla Figura 4.4 come siano stati tutti i primi sample eseguiti a produrre questi valori particolarmente alti in un istanza di tempo circoscritta e consecutiva. La motivazione più plausibile di questa situazione è dovuta ad un picco iniziale nella latenza di rete in quanto successivamente si trovano valori accettabili ed in linea con i restanti benchmark.

Contract - evaluate() Il benchmark visto nella Sezione 4.3.2 con la relativa Figura 4.5, a differenza del primo, presenta una distribuzione molto più uniforme e compatta, con tendenza ad una Normale o Gaussiana, in quanto tutti i valori tendono a concentrarsi attorno ad un singolo valore medio. È una distribuzione ottenibile dalla media, che ne definisce il picco centrale, e dalla deviazione standard che ne definisce la dispersione attorno ad essa. La Figura 4.6 presenta una situazione molto simile a quella del diagramma del precedente benchmark. Sono presenti outliers, più lievi dei precedenti, che però non si trovano all'inizio dell'esecuzione ma oltre il 50esimo sample.

Questa situazione avvalorava maggiormente la nostra ipotesi che la fluttuazione dei risultati dei benchmark e questi outliers dipendano dalle condizioni della rete. Se in tutti i benchmark avessimo trovato un ritardo iniziale allora la causa sarebbe stata molto probabilmente il mancato riscaldamento o la perdita dei valori in cache.

5.2 SQL Benchmark

Verrà effettuato un breve commento sui risultati e sui diagrammi ottenuti dai benchmark di SQL.

Metric - `measure()` Il benchmark visto nella Sezione 4.4.1 con la relativa Figura 4.8 presenta anch'esso una distribuzione abbastanza compatta e simile a una Normale o Gaussiana in quanto tutti i valori tendono a concentrarsi attorno ad un singolo valore medio. La Figura 4.9 rispetto alle due precedenti non presenta evidenze particolari di latenza tranne alcuni outliers distribuiti.

Contract - `evaluate()` Nel benchmark visto nella Sezione 4.4.2 con la relativa Figura 4.10, come nel caso precedente, i valori tendono a concentrarsi attorno ad un singolo valore medio presentando una distribuzione Gaussiana. Si può notare come la densità nella zona di outliers sia la minore rispetto a tutti i benchmark precedenti. La Figura 4.11 presenta un'esecuzione pressoché ottima in quanto sono presenti solo due outliers e gli altri valori si condensano all'interno di un range ristretto. È il benchmark avvenuto con la condizione di rete ideale.

5.3 Valori anomali

Un'aspetto abbastanza particolare notato in entrambi i benchmark, come si può vedere nelle Tabelle 4.1, 4.2 per la SEARCH e nelle Tabelle 4.3 e 4.4 per SQL, è il fatto che l'esecuzione dei metodi `evaluate()` è mediamente 1 ms più breve dell'esecuzione del metodo `measure()`. Nei benchmark di SEARCH *metric* si hanno come valori medi 6.2 ms mentre in quelli *contract* si hanno 4.9 ms. Nei benchmark di SQL *metric* si hanno come valori medi 8.5 ms mentre in quelli di *contract* si hanno 7.6 ms.

Questo risulta essere abbastanza controintuitivo e contro le nostre aspettative in quanto il metodo `evaluate()` contiene al suo interno un'invocazione al metodo `measure()` e dei controlli effettuati sui risultati; logicamente dovrebbe impiegare un tempo maggiore.

Per indagare su questi risultati sono state eseguite altre serie di benchmark. Si è notato come i valori medi di esecuzioni oscillino parecchio tra un test e l'altro riscontrando miglioramenti o regressioni fino al 50% del valore. Le oscillazioni si suppone ancora una volta siano dovute alla latenza introdotta dalla comunicazione di rete in quanto non si presentano in modo prevedibile nei sample. In tutti i test successivi non si è riscontrato nulla di anomalo in merito ai tempi di esecuzione, alle statistiche e ai grafici. Si sottolinea sempre il tempo minore impiegato dalle funzioni `evaluate()`.

L'ultima prova effettuata per dare una spiegazione ai risultati è quella che ci ha fornito maggiori punti su cui riflettere. Abbiamo provato a effettuare i benchmark con l'ordine invertito rispetto alle fasi discusse nel capitolo precedente, sono state benchate prima le funzioni `evaluate()` e poi le `measure()`. È stata provata questa modalità per andare a scartare l'ipotesi che dipendesse dalla cache di Elasticsearch. Si è visto invece, al contrario di come ci si aspettava, che l'ordine con cui vengono effettuati i benchmark va ad influire sulle prestazioni medie finali, ciò è dovuto probabilmente ad un salvataggio dei valori in cache. Elasticsearch utilizza un doppio sistema di caching: da un lato si appoggia sul caching delle pagine del file-system e dall'altro sfrutta un proprio sistema di caching interno contenente i risultati delle query; il secondo benchmark dunque è lievemente favorito rispetto al primo. Si nota come il primo metodo eseguito presenta sempre un rallentamento di qualche decimo di millisecondo.

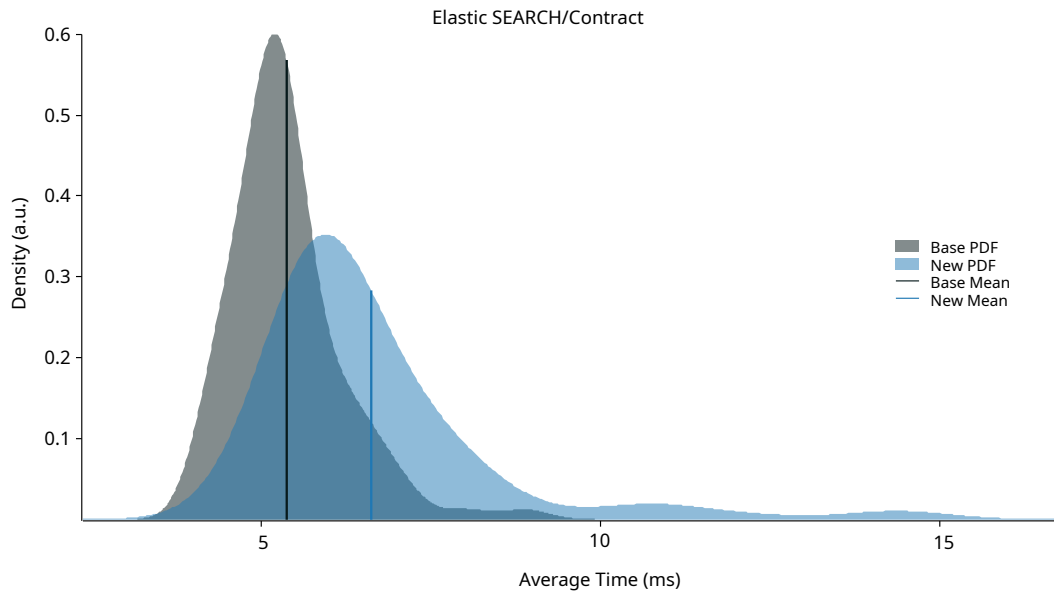


Figura 5.1: Diagramma confronto densità probabilistica richieste `evaluate` di tipo SEARCH

Nella Figura 5.1 viene colorato di grigio la densità del metodo `evaluate()` eseguito dopo il metodo `measure()` mentre in azzurro la densità quando viene eseguito prima. Si nota

come l'area azzurra è più spostata verso destra, risultando in un tempo medio lievemente maggiore.

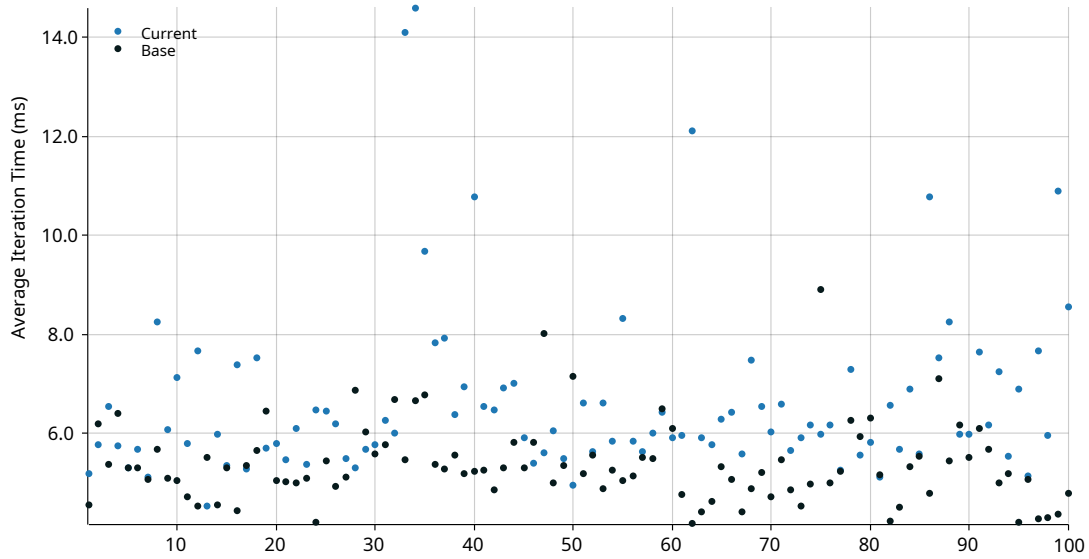


Figura 5.2: Diagramma confronto iteration/time richieste **evaluate** di tipo SEARCH

La Figura 5.2 evidenzia lo stesso concetto mostrando però singolarmente i sample a confronto. I test in cui `contract()` viene eseguito dopo `measure()` sono rappresentati da pallini blu scuro mentre quelli eseguiti invertiti sono rappresentati da pallini azzurri. Si nota anche qua come generalmente i pallini azzurri stiano al di sopra di quelli blu.

In conclusione, si può affermare che l'overhead introdotto dal mancato hit nella cache di Elasticsearch è talmente piccolo che risulta essere trascurabile ai fini della valutazione delle performance, mentre il fattore che causa maggiore oscillazione si dimostra quindi essere la rete e la latenza di trasmissione. Si dimostra anche l'efficienza del sistema, sebbene effettui controlli molto semplici, e si può affermare che l'overhead introdotto dalla computazione risulta essere irrilevante rispetto a quello introdotto dalla rete.

Capitolo 6

Conclusioni

Questa tesi si occupa di soddisfare alcune delle esigenze e limitazioni, introdotte dalla decentralizzazione e dalla conseguente adozione di architetture distribuite basate su microservizi, legate al monitoring.

L'Obiettivo O_1 Estensione *Assurance Engine* di questa tesi, che va a risolvere il primo Gap G_1 Generalizzabilità, è l'estensione di un sistema generico e altamente scalabile, capace di raccogliere dati da diverse fonti e di effettuare valutazioni basate su contratti predefiniti. Viene quindi integrato il sistema presente con Elasticsearch, uno strumento di monitoring, rendendo possibile l'esecuzione di metriche per ottenere *measurements* dal servizio di backend e la valutazione di contratti basati sulle *evidence* ottenute.

Il secondo Obiettivo O_2 Implementazione Contratti, che va a risolvere i successivi due Gaps G_2 Valutazioni performance e G_3 Regole Avanzate per Contratti, riguarda l'implementazione di contratti specifici e più strutturati che definiscono le modalità per verificare PNF sulla base delle *evidence* ottenute. Oltre agli aspetti prestazionali del sistema è possibile la valutazione di proprietà specifiche sui dati ottenuti grazie a delle regole *taylor-made* sviluppate nei contratti, ottenendo così un sistema di *Assurance* più flessibile e completo.

L'ultimo Obiettivo O_3 Valutazione Sperimentale Performance infine viene soddisfatto implementando benchmark che ne valutano l'efficienza complessiva.

6.1 Metodologia e Funzionamento

La *metodologia di Assurance* adottata, illustrata nel Capitolo 2.4.1, si basa sulla definizione di due componenti base dell'intera infrastruttura, le *metriche* e i *contratti*, per la verifica di PNF, ovvero vincoli al modo in cui il sistema implementa e fornisce le

sue funzionalità. I dati vengono raccolti dagli endpoint del sistema target attraverso l'esecuzione di Metriche focalizzate sulla misurazione di un comportamento specifico. Una volta raccolte le *evidence* o prove, i Contratti ne specificano come deve avvenire la valutazione.

Il *processo di Assurance* utilizzato, illustrato nel Capitolo 2.4.2, si suddivide in 4 fasi eseguite ciclicamente. *Evaluation mapping*, identificazione delle metriche e dei contratti necessari per verificare le proprietà desiderate. *Measurement collection*, valutazione delle metriche e salvataggio delle evidence. *Contract evaluation*, calcolo del risultato booleano dei contratti applicati alle evidence raccolte sulla base delle condizioni espresse al loro interno. *Report composition*, i risultati vengono aggregati in un report machine-readable che contiene l'esito della verifica e le evidence su cui si basa.

Il *funzionamento dell'Assurance-Engine*, illustrato nel Capitolo 3.4.5, prevede che un utente interagisca con il metodo POST di una delle API esposte per dare inizio all'esecuzione. L'esecuzione rispecchia la suddivisione fatta nel processo di Assurance e ne vengono illustrati i 3 punti principali. *Measurement collection*, viene utilizzato il metodo `measure()` per ottenere prove da un target o per inviare richieste personalizzate; i dati raccolti vengono identificati come *measurements* e costituiscono le *evidence* necessarie per la valutazione del sistema. Si esegue infine un parsing della risposta JSON per estrarre campi e valori desiderati. *Contract evaluation*, vengono eseguiti controlli booleani, tramite il metodo `evaluate()`, confrontando le *measurements* ottenute con i parametri del contratto; i risultati dei vari controlli vengono infine aggregati per stabilire se la proprietà oggetto di verifica è stata soddisfatta. *Report composition*, i risultati finali vengono formalizzati in una struttura contenente le evidence, il risultato e il timestamp per poi essere inviate in formato JSON all'utente che ha avviato la richiesta.

6.2 Esperimenti e risultati

Come mostrato nel Capitolo 3 e verificato nel Capitolo 4 sono stati condotti 2 benchmark per valutare le performance del sistema sviluppato sull'ottenimento di metriche e sulla valutazione di contratti. Il primo benchmark esegue query nel Query DSL di Elastic mentre il secondo le esegue in SQL. Entrambi valutano le performance di esecuzione dei metodi `measure()` ed `evaluate()`, rispettivamente di Metriche e Contratti. I risultati emersi hanno dimostrato una grande efficienza del sistema, eseguendo le query proposte in tempi tra i 6 e gli 8 millisecondi.

Data la differenza anomala di tempistiche tra i due metodi, illustrata nella Sezione 5.3, si è notato come Elasticsearch performi leggermente meglio quando possiede le query e i risultati in cache, con miglioramenti nell'ordine dei decimi di millisecondo, trascurabili in termini di valutazione. Si sono notate anche oscillazioni nelle performance tra benchmark eseguiti consecutivamente con regressioni e miglioramenti fino al 50%, rientrando

però sempre in un range di efficienza accettabile. Analizzando i sample dei benchmark è stata associata alla rete la causa principale di questa latenza variabile, in quanto non si presenta in modo prevedibile tra le richieste.

Il sistema sviluppato permette quindi di effettuare *verifiche di Assurance* tramite dati collezionati all'interno di Elasticsearch con estrema efficienza e poco overhead. Si è notato come il tempo computazionale di esecuzione dei metodi `measure()` o `evaluate()`, i quali assolvono rispettivamente al compito di ottenere prove e verificare contratti complessi per garantire PNF, sia proporzionalmente irrisorio rispetto alla latenza introdotta dalla rete che ne definisce il tempo medio di esecuzione. Di conseguenza, si ottiene un sistema estremamente efficiente che si pone come intermediario generando un overhead trascurabile.

6.3 Future Work

In questa tesi sono state poste le basi per un *sistema di assurance* per sistemi distribuiti scalabile e generico. Il sistema attualmente realizzato si presta come struttura di partenza per nuovi possibili sviluppi e migliorie.

Nuove integrazioni Il framework descritto, grazie alla sua struttura modulare e alla funzione di intermediario, può essere facilmente esteso per integrare nuove sorgenti di dati oltre a quelle già supportate, come Elasticsearch e Prometheus, e per eseguire specifiche *probe ad-hoc*. Questo consente la creazione di un sistema unificato in grado di interfacciarsi con molteplici soluzioni di monitoring e storage comunemente utilizzate. L'obiettivo finale è sviluppare una piattaforma centralizzata in grado di raccogliere e gestire dati da diverse fonti, migliorando la visibilità e il controllo su vari sistemi informativi. La modularità è il fattore chiave di questa soluzione che ne permette una facile estensibilità e una maggiore versatilità nel rispondere a esigenze diverse.

Contratti e probe avanzate Attualmente, le API esposte dal sistema offrono solo una gamma limitata di funzionalità. Una direzione di sviluppo promettente è l'espansione delle API esposte per supportare funzionalità più complesse e specifiche. Tra questi sviluppi futuri ricade la generazione di nuovi contratti, sempre più strutturati e complessi, sui dati raccolti dalle varie sorgenti per verificare PNF. Allo stesso modo, si possono sviluppare probe che eseguono test più sofisticati su comportamenti dei sistemi target. L'obiettivo è ottenere un sistema più robusto, che permetta di effettuare una vasta gamma di controlli, sempre più fini, sulla base del target scelto per verificare proprietà specifiche.

Contratti su sistemi reali Finora, il sistema è stato testato in ambienti controllati o sperimentali, il cluster del laboratorio, dove si è comportato come previsto. La fase successiva consiste nell'andare ad applicarlo ad un caso di studio reale, monitorando un sistema di produzione con traffico e carichi effettivi; è un passaggio cruciale per verificare come il framework gestisce situazioni reali, incluse eventuali anomalie o casi limite non riscontrati negli ambienti sperimentali. Inoltre, ciò consentirà di eseguire un'analisi comparativa tra i benchmark prestazionali effettivi e quelli sperimentali, facilitando l'individuazione di aree di ottimizzazione e potenziali miglioramenti.

Scalabilità Il sistema è stato progettato per essere scalabile, senza necessità di amministrazione aggiuntiva o overhead. Per verificare effettivamente quanto bene il framework possa scalare, si possono eseguire una serie di test che misurano le performance ottenute con diverse repliche deployate dell'assurance-agent. Questo tipo di test fornirà dati concreti sulle performance di un sistema scalato su varie istanze e permetterà di verificare o meno l'assunzione teorica della facilità di scalabilità fatta nella sezione sulla replicabilità.

Bibliografia

- [1] A. S. Tanenbaum and M. v. Steen, *Distributed Systems: Principles and Paradigms (2nd Edition)*. USA: Prentice-Hall, Inc., 2006.
- [2] F. Ponce, G. Márquez, and H. Astudillo, “Migrating from monolithic architecture to microservices: A rapid review,” in *2019 38th International Conference of the Chilean Computer Science Society (SCCC)*, 2019, pp. 1–7.
- [3] Y. Wei and M. B. Blake, “Service-oriented computing and cloud computing: Challenges and opportunities,” *IEEE Internet Computing*, vol. 14, no. 6, pp. 72–75, 2010.
- [4] S. Ben Atitallah, M. Driss, and H. Ben Ghezala, “Microservices for data analytics in iot applications: Current solutions, open challenges, and future research directions,” *Procedia Computer Science*, vol. 207, pp. 3938–3947, 10 2022.
- [5] A. El Akhdar, C. Baidada, and A. Kartit, “Adaptability of microservices architecture in iot systems: A comprehensive review,” in *Proceedings of the 7th International Conference on Networking, Intelligent Systems and Security*, ser. NISS '24. New York, NY, USA: Association for Computing Machinery, 2024. [Online]. Available: <https://doi.org/10.1145/3659677.3659734>
- [6] A. Balalaie, A. Heydarnoori, and P. Jamshidi, “Microservices architecture enables devops: Migration to a cloud-native architecture,” *IEEE Software*, vol. 33, no. 3, pp. 42–52, 2016.
- [7] F. Berto, “Assurance-aware 5g edge-cloud architectures for intensive data analytics,” Ph.D. dissertation, università degli Studi di Milano, Milan, January 2024. [Online]. Available: <https://air.unimi.it/handle/2434/1021895>
- [8] S. Moreschini, F. Pecorelli, X. Li, S. Naz, D. Hästbacka, and D. Taibi, “Cloud continuum: The definition,” *IEEE Access*, vol. 10, pp. 131 876–131 886, 2022.
- [9] N. Z. Sherif B. Azmy, Rawan F. El-Khatib and H. S. Hassanein, “Extreme edge

computing challenges on the edge-cloud continuum,” Queen’s University, Kingston, ON, Canada, Tech. Rep., 2024.

- [10] H. Siddiqui, F. Khendek, and M. Toeroe, “Microservices based architectures for iot systems - state-of-the-art review,” *Internet of Things*, vol. 23, p. 100854, 2023. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S2542660523001774>
- [11] ISO Central Secretary, “System and software Quality Requirements and Evaluation (SQuaRE),” International Organization for Standardization, Geneva, CH, Standard ISO/IEC 25010:2023, 2023. [Online]. Available: <https://www.iso.org/standard/78176.html>
- [12] C. A. Ardagna, N. Bena, and R. M. de Pozuelo, “Bridging the gap between certification and software development,” in *Proceedings of the 17th International Conference on Availability, Reliability and Security*, ser. ARES ’22. New York, NY, USA: Association for Computing Machinery, 2022. [Online]. Available: <https://doi.org/10.1145/3538969.3539012>
- [13] C. A. Ardagna and N. Bena, “Non-functional certification of modern distributed systems: A research manifesto,” in *2023 IEEE International Conference on Software Services Engineering (SSE)*, 2023, pp. 71–79.
- [14] ISO Central Secretary, “General requirements for the competence of testing and calibration laboratories,” International Organization for Standardization, Geneva, CH, Standard ISO/IEC 17025:2017, 2017. [Online]. Available: <https://www.iso.org/standard/66912.html>
- [15] A. Hudic, M. Tauber, T. Lorünser, M. Krotsiani, G. Spanoudakis, A. Mauthe, and E. R. Weippl, “A multi-layer and multitenant cloud assurance evaluation methodology,” in *2014 IEEE 6th International Conference on Cloud Computing Technology and Science*, 2014, pp. 386–393.
- [16] M. Anisetti, C. A. Ardagna, F. Gaudenzi, and E. Damiani, “A certification framework for cloud-based services,” in *Proceedings of the 31st Annual ACM Symposium on Applied Computing*, ser. SAC ’16. New York, NY, USA: Association for Computing Machinery, 2016, pp. 440–447. [Online]. Available: <https://doi.org/10.1145/2851613.2851628>
- [17] M. Anisetti, C. A. Ardagna, E. Damiani, and F. Gaudenzi, “A semi-automatic and trustworthy scheme for continuous cloud service certification,” *IEEE Transactions on Services Computing*, vol. 13, no. 1, pp. 30–43, 2020.

Bibliografia

- [18] M. Anisetti, C. A. Ardagna, N. Bena, and A. Foppiani, “An assurance-based risk management framework for distributed systems,” in *2021 IEEE International Conference on Web Services (ICWS)*, 2021, pp. 482–492.
- [19] M. Anisetti, C. A. Ardagna, N. Bena, and R. Bondaruc, “Towards an assurance framework for edge and iot systems,” in *2021 IEEE International Conference on Edge Computing (EDGE)*, 2021, pp. 41–43.
- [20] N. Bena, R. Bondaruc, and A. Polimeno, “Security assurance in modern iot systems,” in *2022 IEEE 95th Vehicular Technology Conference: (VTC2022-Spring)*, 2022, pp. 1–5.
- [21] M. Anisetti, C. A. Ardagna, and N. Bena, “Multi-dimensional certification of modern distributed systems,” *IEEE Transactions on Services Computing*, vol. 16, no. 3, pp. 1999–2012, 2023.